AQA A Level Computer Science NEA

Shayan Nezami

March 12, 2026

Contents

List of Figures

Listings

1 Analysis

1.1 Introduction

A backup utility is an essential software program for any computer system, allowing users to keep their files and data safe in the case of hardware failure, or various possible software issues. It is particularly important that these files are stored externally, on another device, in case of such hardware failure. Therefore, I will create a backup utility for storing files on an external server from a client PC. This will require various components, such as encrypting the files stored on the server, encrypting the transmission of files between client and server, and compressing the files being transferred.

The main benefits of the program I will create over existing systems are that my utility will be fully open source, so the function of the program can be independently inspected by any user, and that it will be tailored to my client's exact needs: many existing applications are either closed source, or too bloated with unnecessary features and unintuitive interfaces. Instead, this will be a simple, easy-to-use command line tool.

I will research this through two main methods: a discussion with my client, and using reliable external sources. The latter may come from reputable books, websites, papers, or journals, and will be cited.

1.2 Client Overview

My client will be a classmate – Sam Yarasani – who informed me he is looking for such software. Therefore, I will tailor this program for his exact needs, and I will establish his specific requirements through a client interview. This has the benefit over existing systems of being in exactly the format he would like, without additional features which add complexity and bloat to the final program. Plus, it will have the exact user interface he would like.

1.3 Client Interview

Below is a record of the transcript of my first interview with my client. I will use this to inform the design requirements for this backup utility.

Q: I understand that you are looking for a backup utility?

A: Yes, I'd like to be able to create external backups of my system on a separate device.

Q: What devices would you like to create backups of?

A: My laptop and desktop PC.

Q: What operating system are those devices running?

A: Arch Linux, and Ubuntu 22.04 respectively, but I would like this to work on any modern Linux distribution - I may want to change in the future!

Q: Where would you like the backups to be stored?

A: I have a fairly old laptop which I use to run a few home server utilities. I'd like to also store the backups here, if possible.

Q: Can you be more specific about the laptop?

A: It's around 15-20 years old, I'm not exactly sure, with 4GB of RAM. Also, it's running Ubuntu server.

Q: Would you like a third-party server in a data centre to store the backups instead?

A: Absolutely not. The key here is that this is all fully local to me, and that I am the only one with access to the backups. Many current backup utilities lack this!

Q: Would you want the files stored to be encrypted in transmission, and while stored?

A: It is crucial for the files to be hidden while being transferred, as I do not want any data being intercepted by others. However, I don't need the files to be encrypted while being stored on the server, as I own that server and know that it is kept safe.

Q: Do you have any other priorities?

A: File transfers should be as quick as possible, and as little space as possible should be used on the server device. Oh, and I want to be sure that transfers have been successful.

Q: What type of files will you be backing up?

A: Well, I will be using this for my whole system, so all sorts of files. I guess there will be text files, like system configuration files, image files, video files, and raw binary executables.

Q: Okay. The non-text files you were talking about, are they already compressed?

A: Yes actually, almost all of my audio and video files are already compressed.

Q: Very good, thanks. What about the user interface, what form should this take?

A: I want a very simple command line interface, with no messy graphical elements. It could have an interactive mode, but there should certainly be a way to use it non-interactively, with just command line flags, so that I can use it in scripts.

Q: Finally, is it important to you for this project to be finished by a certain date?

A: Absolutely. It is essential for this to be finished by the end of March 2026. This is one of my key priorities, since I will start work on an important project in April, and I need to ensure I have safe backups of my files by its start.

I have summarised my key findings from this interview below:

- The program should work on Linux systems,
- The program should be able to comfortably run on my client's devices, without putting much strain on them,
- I must also code the server storing the files, which should run on a device my client also owns,
- The files must be encrypted in transmission,
- The files should be compressed before transmission, and decompressed upon retrieval by the client, so that the transmission is as quick as possible, and as little space as possible is taken up on the server,
- Only text files need to be compressed, as most other files are already compressed,
- There should be no graphical user interface, but instead a command-line interface that can be used non-interactively,
- There is a hard deadline of the 31st March 2026 for finishing the project.

1.4 Specification

Based on the requirements established in the client interview, I have established the following specification points. These will guide me throughout the development process, and can serve as a metric against which the final product is evaluated.

1. The program should allow the user to create file backups on a server, run by the client themselves,
    (a) The software should be able to run on both the client and server, with appropriate functionality for each situation,
    (b) They should be able to work together, with a server being able to store the user's backed up files,
2. The client program should run on modern Linux systems, usable on a laptop or desktop computer,
    (a) An installable binary package should be created that is executable on amd64 Linux systems,
    (b) The client should not at any point use more than 500MB of RAM,

   (c) The client should never use more than 5% of CPU time while running on a modern laptop; my client's processor, the Intel i9-12950HX, can be used for a test case,

3. It should be usable with an intuitive command-line interface, not requiring interactive input,
   (a) All required functions of the program should be able to be passed in through command line flags and parameters,
   (b) There can be an optional interactive mode, but this should not allow any functionality than the non-interactive mode does not,
   (c) There should be no graphical interface,
4. The server should be able to run on a low-end laptop 10-15 years old,
   (a) The program should avoid reading and writing to the system drives as much as possible, since most such devices use HDDs,
   (b) The server program should not at any point use more than 250MB of RAM
   (c) The server program should never use more than 10% of CPU time while running on such a device; my client's desired server's processor, a 2nd generation Intel i5 processor, can be used as a test case,
5. All file transfers should be secure, so a man-in-the-middle would not be able to intercept data,
   (a) Any data intercepted should be of a format that the interceptor would not be able to decipher, or gain any useful information from,
   (b) The methods used should have their correctness checked,
6. Text files stored on the server should be compressed such that they use a reasonable amount of storage,
   (a) They should be at most as large as the uncompressed files, i.e. file size should never increase following compression,
   (b) The client should be able to decompress any file when needed by the user, storing any needed details about the compression within the file,
7. File transfers should be verifiably correct, so data is not corrupted during transmission,
   (a) Once a file has been transferred to the server, it should be compared with the original file on the client device,
   (b) This checking should be with a much smaller version of the file than that which was originally sent for most files – it should be up to 10kB in size,

1.5 Technical Points

Based on the specification points established, I have extracted the key technical challenges that will be involved in building the software below.

1. Implement a transmission system with asymmetric encryption for key exchange between the client and the server,
2. Implement symmetric encryption between the client and the server after keys have been established,
3. Use a method to verify the integrity of the files transferred,
4. Implement a lossless compression algorithm for the text files stored on the server.

1.6 Program Structure Diagram

This diagram (Figure 1) shows how these main technical blocks will interact with each other in the client and server programs. While they will run different main procedures, these will share some key components.

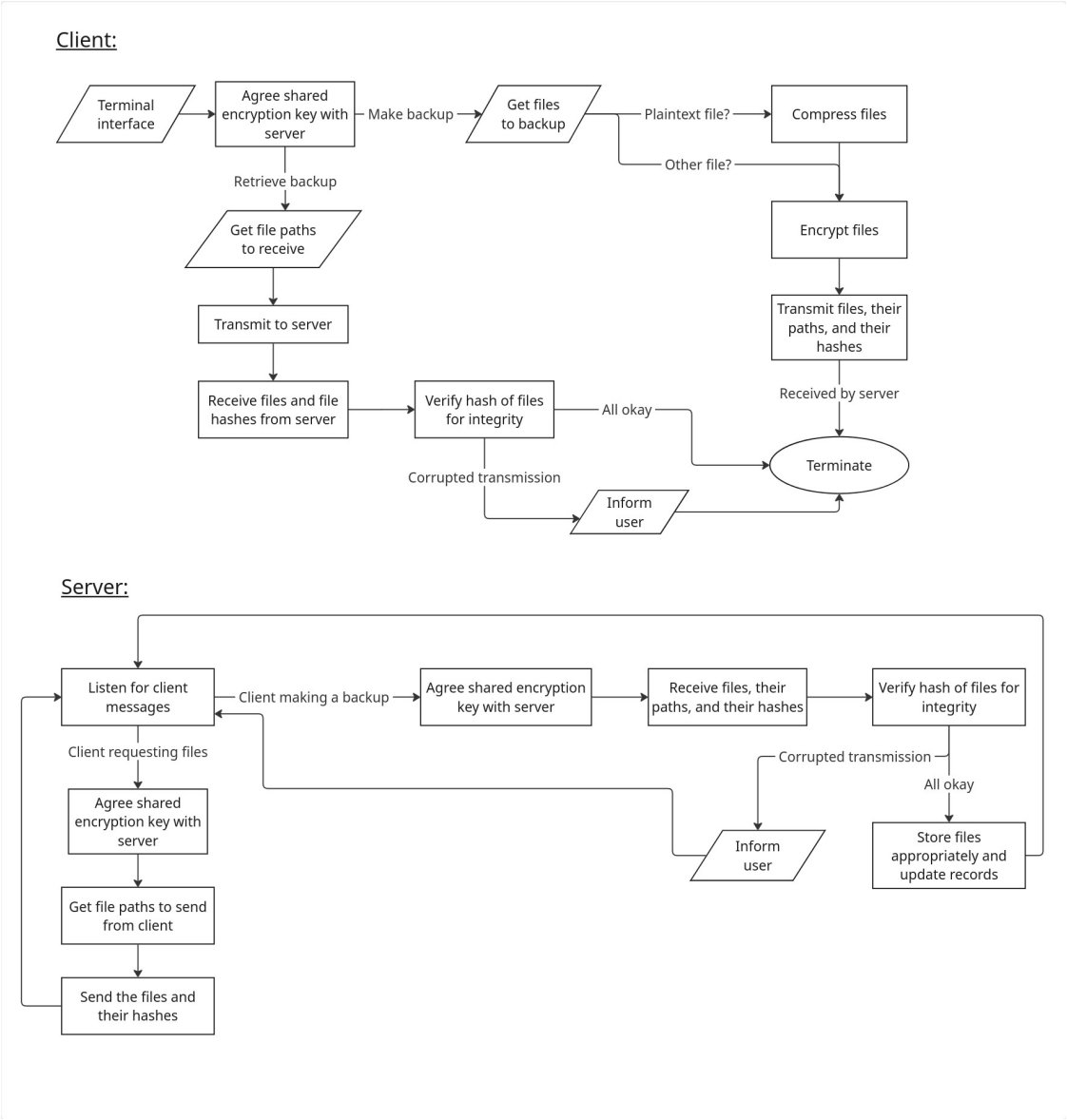

Figure 1: Program Structure Diagram

I will now consider how I can implement these main blocks.

1.7 Transmission Procedure

This is one of the largest components of the program, which is used for any and all communications between the client and server devices. There are some key points that need to be individually considered:

1. There needs to be a procedure used to exchange keys between the sender and receiver, perhaps in a similar style to the RSA algorithm, providing asymmetric encryption,
2. There must be a symmetric encryption algorithm used to encrypt communications following the establishment of keys,
3. The communications should be verified to be correct, using a method to check that there has not been any corruption during data transfer.

1.7.1 Key Exchange

The three algorithms I will consider for this are RSA, Diffie-Hellman Key Exchange, and Elliptic Curve Cryptography.

These algorithms are commonly used, and very well documented. All three rely on mathematical problems that are intractable to solve without some prior knowledge: RSA relies on the difficulty in factoring a large semiprime integer (one with exactly two prime factors), while Diffie-Hellman and elliptic curve systems rely on the discrete logarithm problem.

While elliptic curve cryptography can provide equivalent security with much smaller keys, and requires less processing power for the sender and receiver, this is unlikely to be at all impactful on the small scale of this backup utility, where this asymmetric encryption is just used to share a symmetric encryption key. However, these algorithms are tough to implement, and involve very high-level mathematics. This could increase the time taken to deliver the program past the client's deadline, and adds more avenues for potential bugs to develop. This can be very dangerous, as shown in the following paper [1], where a small error in the implementation led to researchers being able to extract the full key.

Plus, implementing it could cause me to fall short of my deadline, and it is very important for my client that I do not. Therefore, I will exclude elliptic curve cryptography from my choices.

I will now consider the basic implementation details of the RSA and Diffie-Hellman algorithms to aid my decision-making.

RSA

The website [2] contains details about the mathematics of the RSA algorithm, and how it can be implemented. From this, I have generated the following steps that are taken in the RSA algorithm.

1. The receiver generates two large primes, p, q, then computes their product, $n = pq$.
2. Euler's Totient function, $\phi(pq) = (p-1)(q-1)$, is applied to n.
3. An encryption exponent, e, is selected such that $1 < e < \phi(n)$, and e is relatively prime with $\phi(n)$. Note that $a, b \in \mathbb{Z}^+$ are relatively prime $\iff \nexists\, x \in \mathbb{Z} : x > 1,\ x \mid a$, and $x \mid b$.
4. A decryption exponent, d, is calculated such that $de = 1 \pmod{\phi(n)}$. The extended Euclidian algorithm provides a method for this.
5. The public key is formed from (n, e) and the private key (n, d). The receiver sends their public key only to the sender.
6. The encrypted ciphertext, C, is calculated through $C = M^e \pmod{n}$, where M is some numerical representation of the message.
7. The ciphertext is decrypted into the plaintext message, M, by the calculation $M = C^d \pmod{n}$.

Now, the sender encrypts their message with the receiver's public key, and then only the receiver can decrypt it since only they have the private key. Computing $\phi(n)$ is infeasible without knowing the primes, p, q, that n is a product of, and this is also infeasible to compute due to the large size of the primes. This means no other party can calculate the private key from the public key.

Diffie-Hellman key exchange

Page 84 of the following lecture notes [3] contains a brief description of the mathematics behind the Diffie-Hellman key exchange. Below I describe the key steps in this algorithm in my own words.

1. Either the sender or the receiver randomly generates a large prime, p, and a large number, g, and sends these to the other.
2. The sender and receiver then generate random secrets s_a and s_b respectively, for $s_a, s_b \in \mathbb{Z}^+$.
3. The sender sends $a = g^{s_a} \pmod{p}$ to the receiver, who computes $a^{s_b} = g^{s_a s_b} \pmod{p}$.
4. The receiver sends $b = g^{s_b} \pmod{p}$ to the receiver, who computes $b^{s_a} = g^{s_a s_b} \pmod{p}$.
5. Now, they both have the shared secret, $S = g^{s_a s_b} \pmod{p}$, which can be used to establish a shared key.

An attacker would have access to $g,\ p,\ g^{s_a} \pmod{p},\ g^{s_b} \pmod{p}$, and need to compute $S = g^{s_a s_b} \pmod{p}$ to compromise the conversation. A quick solution to the Discrete Logarithm Problem would imply a quick solution to this problem, but one is not currently known. This is central to the security of this key exchange, similarly to the prime factorisation problem for RSA.

As can be seen, RSA and Diffie-Hellman are very similar in security, and difficulty of implementation, however the advantage of RSA is that it can also be used to provide signed certificates to

prove the authenticity of the sender. This has not been an aim as of yet, but using RSA will make it easier to implement this if desired in the long-term, since I intend to make this project open-source upon completion.

1.7.2 Symmetric Encryption

Following the establishment of keys, it is greatly advantageous to use a symmetric encryption system rather than continuing with asymmetric systems, since these are much faster and more efficient. A similar process is used by the TLS protocol, which secures more internet communications.

There are a great number of available symmetric encryption algorithms, including the Caesar Cipher, AES, ChaCha20, and Blowfish, among others.

The Caesar cipher can be immediately eliminated, since it is trivial to break with even very simple cryptanalysis. However, the other algorithms are very similar in that they all provide secure encryption that is very tough to break. The final choice of algorithm should be one that is well-documented, simple to implement, and efficient to run.

However, AES has been used as the basis of internet security for decades, and as such there has been great research into verifying its security, and ensuring implementations do not contain vulnerabilities. Plus, C# provides a standard implementation of this. Therefore, I will use AES for my symmetric encryption needs.

1.7.3 Verifying the Integrity of Communications

It is necessary to ensure all communications have arrived at the desired recipient uncorrupted, exactly as they were sent. Therefore, some method must be used to ensure that this is the case.

Some simple methods for this include majority voting, where each bit is sent an odd number (> 1) of times, and the most common bit is accepted, and parity bits, where a bit is added to the end of each string of defined length to ensure that the number of 0s or 1s is either even or odd as desired.

However, these methods allow multiple mistakes to "cancel" each other out, and say little about the correctness of the whole message. A more robust method is to compute some form of hash from the message, and send that directly after it. The receiver will also compute a hash of the data received, and compare it to the hash sent. If these match, then the data has arrived intact. This verifies that the whole message has arrived safely, while requiring comparatively little additional data to be transferred. Plus, these algorithms can run very quickly.

I can use the SHA-256 algorithm for this. It is a cryptographically secure hash function, that produces a 256-bit hash for any length of text. It is widely used, and it is very unlikely for two distinct inputs to give the same output. There is also an available C# implementation that I can use for this.

1.8 Compression

Firstly, it is important to establish that I will want to use a lossless algorithm, since a backup program should not change the files being backed up – the whole point is to provide a safe copy of the actual system files themselves. Plus, lossy compression of important system files would be catastrophic.

There is also the key problem that many files being backed up will already be compressed. The system must be able to deal with these, not producing output with larger file size than the uncompressed versions.

There is also the important consideration of whether each file is individually compressed, or whether the whole block of files is compressed together.

There are two broad options I can take based on the above. I could either compress the whole block of data, or each file individually, with some metadata at the beginning of the file outlining what procedure (if any) was used.

The first option will not be very useful, since different file types can be effectively compressed using vastly different algorithms, so I will not be able to achieve significant size reductions in this way. Therefore, I believe the right choice is to compress files individually.

This means that I would need to write different algorithms for different file types, and have a way of detecting files that have already been compressed. However, the discussion with my client revealed that he already compresses the majority of non-text files on his system. So, I have decided to just implement compression for plaintext files, where the most space savings can be made, while minimising the unnecessary processing taking place, so the program executes as quickly as possible. This will also reduce development time, allowing me to finish the program in time for my client's strict deadline.

Three lossless plaintext compression algorithms I will consider as the basis of my algorithm are Run-Length Encoding, Huffman Coding, and Prediction by Partial Matching.

Run-Length Encoding works by representing repeated runs of one character as frequency-data pairs. For example, AAAAAABBB would be stored as "6A3B". However, this is not very effective for text files, where there is not often repeated runs of characters; it is instead very useful for images, where there may be, for example, a dark background with many long runs of dark pixels. Therefore, it is unsuitable for my needs, to compress plaintext.

Prediction by Partial Matching uses complex statistical techniques to predict the most probable next characters based on the previous ones, assigning more probable characters shorter binary representations. However, to create and optimise such an algorithm, it would take a long time, which may cause be to pass my client's non-negotiable deadline. Plus, it is more computationally complex than the alternatives, so the compression and decompression algorithms would use more memory and processor time, and would likely take longer to run.

Huffman Coding provides a perfect balance between complexity and how effective it is at compressing plaintext. It looks at the file as a whole, assigning more frequently used characters shorter binary representations. This is very effective for text files, where some letters are used

more often than others. Moreover, most available Unicode characters are never used (special characters, scientific symbols, foreign language characters...) in most files, so the majority of characters would have a shorter representation in a file compressed in this way. While it may be more complex than Run-Length Encoding, it is perfectly feasible for me to implement it in the given time, and the implementations can be very quick! Even though Prediction by Partial Matching can in some cases provide better compression, for the files where this really matters to my client, they can just compress them themselves before backing them up.

I have researched the Huffman Coding algorithm through this website [4], which gives an explanation of one implementation of it.

The high-level steps of the encoding algorithm are as follows:

1. Count the number of times each character appears in the text,
2. Turn each character into a tree with a single node, weighted by its frequency,
3. Continually merge the two nodes with the smallest weights into a new tree, creating a new node with the nodes being merged as children, with its weighting being the sum of its two children's weights,
4. Stop once there is only one tree left - this is the Huffman tree,
5. Carry out a post-order tree traversal on the Huffman tree to write a representation of it to the beginning of the compressed file, and to build up a mapping of characters to their binary Huffman representation (this is defined by their placement in the tree with respect to the root node),
6. Go through each character in the file, writing its Huffman representation to the compressed file.

The high level steps of the decoding algorithm are as follows:

1. Read the Huffman tree from the start of the compressed file, representing it as a tree in the program,
2. Carry out a post-order traversal of the tree to build up a mapping of the Huffman binary bit-strings to the characters they represent,
3. Go through all of the bits in the compressed file, writing the character they represent to the decompressed file.

1.9 Class Diagram

Draw one

1.10 Order of Completion

This is a rough timeline of how long I expect it would take me to complete the sections outlined above.

1. Plaintext compression - 2 weeks
2. RSA style key exchange - 2 weeks
3. Hashing to verify transmission, and symmetric encryption to secure it - 1 week
4. Finalising the transmission system - 2 weeks
5. Terminal interface - 1 week
6. Putting it all together - 2 weeks

In total, this will take 10 weeks, leaving me on track to meet my deadline with plenty of time to spare in case unexpected issues arise.

2 Documented Design

2.1 Huffman Compression

As decided upon in my Analysis (Section 1.8), I will use Huffman Coding to compress all plaintext files that are being backed up. The compression and decompression will occur on the client-side, before the files and sent, and after they are received.

The benefits of this are twofold. Firstly, this means less data is transmitted, reducing the risk of corruption, and speeding up the backup process. Secondly, as identified in my Client Interview (Section 1.3), my client will run the server on hardware that is significantly less powerful than their laptop, which acts as the client. Therefore, it is advantageous to run as much processing as possible on the client-side.

2.1.1 Encoding and Decoding

This website [4] provides a thorough written explanation of the Huffman algorithm. Using this to better understand the process, I have created a flowchart depicting how I can implement the encoding (Figure 2), and the decoding (Figure 5).

These flowcharts include implementation details, such as how dictionaries, stacks, priority queues, and recursive subroutines are used to carry out some of the high-level tasks described in the source. My implementation also ensures the compression does not (significantly) increase the size of the files, by using a 1-bit flag at the start of the file to indicate whether compression has been used, and not using any compression if it would increase the file size overall. Do note that the final file will still be one byte larger in this case due to the extra bit added, but this is insignificant and unavoidable.

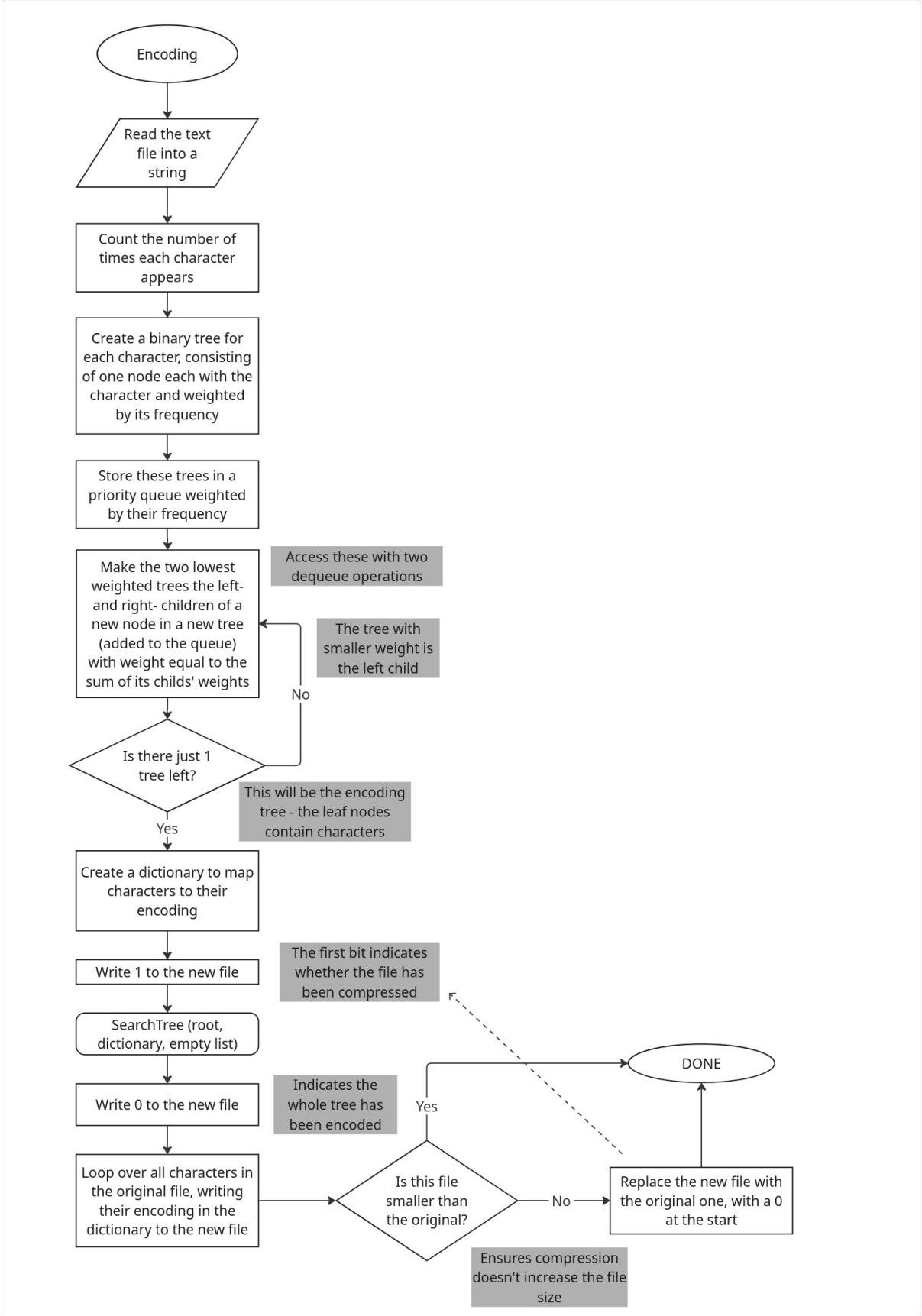

Figure 2: A flowchart depicting my Huffman encoding procedure

In this encoding procedure, the recursive subroutine SearchTree is called. Figure 3 provides a flowchart showing its function.

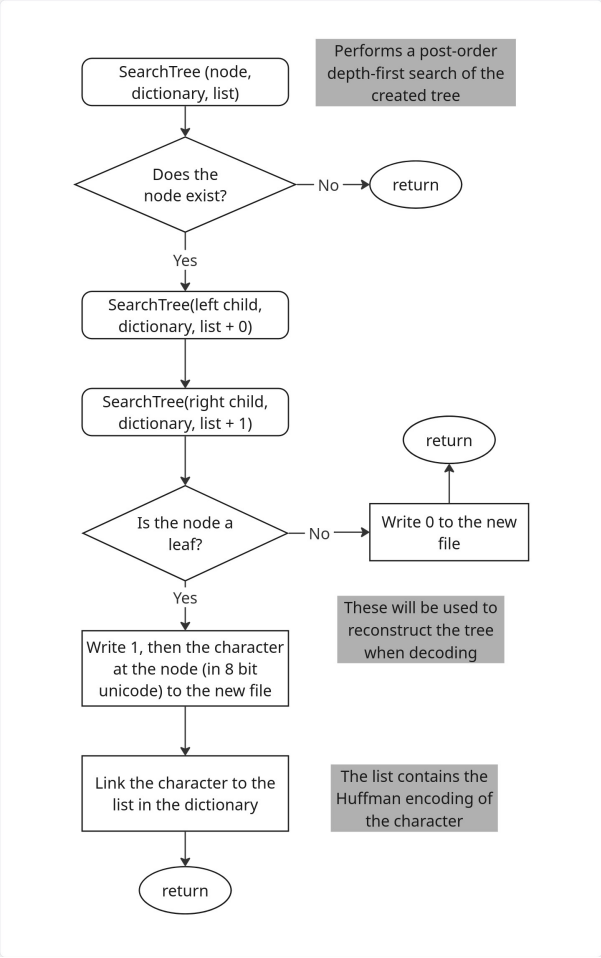

Figure 3: A flowchart depicting my SearchTree procedure

It performs a post-order depth first search of the binary Huffman tree created. By searching the tree in this way, when each node is visited it is written to the compressed file, since the tree needs to be re-read by the decryption program to allow the file to be decrypted. If a leaf node is read, the character belonging to the node is written to the file (in 8-bit Unicode) after a 1 bit. If the node is not a leaf (and so does not contain a character), this is denoted by a 0 being written to the file. This is sufficient for the decoder to fully reconstruct the tree.

The procedure also links each character to its Huffman encoding as the tree is searched (going left in the tree adds a 0 to the encoding, going right adds a 1). This is used later in the program to write each character's Huffman encoding to the compressed file. Filling the dictionary in this procedure means that another search of the tree is not needed, making the code quicker to run.

The code for this procedure can be found in Listing 1.

```
1 // fill byteMaker with encoded tree, and CharacterEncodings with bits
     encoding characters, uses post order DFS
2 private static void SearchTree(HuffmanNode tree, ByteMaker byteMaker,
     Dictionary<char, List<int>> CharacterEncodings, List<int> current)
3 {
```

```
4       if (tree == null) return;
5
6       SearchTree(tree.GetLeft(), byteMaker, CharacterEncodings,
            current.Append(0).ToList());
7       SearchTree(tree.GetRight(), byteMaker,  CharacterEncodings,
            current.Append(1).ToList());
8
9       if (tree.HasCharacter()) // is leaf
10      {
11          byteMaker.Add(1);
12          byteMaker.Add(tree.GetCharacter());
13
14          CharacterEncodings[tree.GetCharacter()] = current;
15      }
16      else
17      {
18          byteMaker.Add(0);
19      }
20  }
```

Listing 1: Code for the SearchTree procedure

This tree that is searched in the above procedure is created using a priority queue to continually merge trees, until just one is left. The snippet of the flowchart that relates to this section of the code can be found in Figure 4.

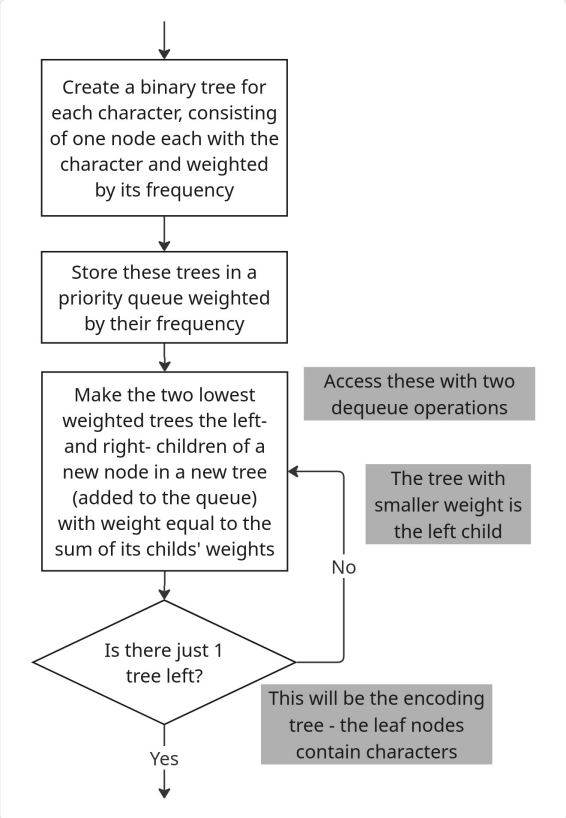

Figure 4: A flowchart depicting the creation of the Huffman tree

First, nodes are created for each character, and these become trees of size 1. They are placed in a priority queue such that the nodes with the greatest frequencies are last. Then, the nodes with the two lowest frequencies are continually dequeued, and made children of a new node, which is enqueued onto the priority queue. Since this is not a leaf node, it does not contain a character. This node's frequency is equal to the sum of the frequencies of its children. The less frequent node is made the left child of the tree. This repeats until just one tree remains, which is the final Huffman tree, which can be used to generate encodings for each character.

The code for this section can be found in Listing 2.

```
// place characters in priorityqueue (based on freq) to generate the tree
PriorityQueue<HuffmanNode, int> NodeQueue = new PriorityQueue<HuffmanNode, int>(); // "base" trees of size 1

foreach (KeyValuePair<char, int> kvp in CharacterCounts)
{
    NodeQueue.Enqueue(new HuffmanNode(kvp.Key, kvp.Value), kvp.Value);
}

HuffmanNode tree = MergeNodes(NodeQueue); // tree = top node of tree

// recursively merge nodes to make a tree
private static HuffmanNode MergeNodes(PriorityQueue<HuffmanNode, int> trees)
{
    if (trees.Count == 1) return trees.Dequeue();

    HuffmanNode tree1 = trees.Dequeue();
    HuffmanNode tree2 = trees.Dequeue();

    int newFreq = tree1.GetFrequency() + tree2.GetFrequency();

    HuffmanNode newTree = new HuffmanNode(tree1, tree2, newFreq);

    trees.Enqueue(newTree, newFreq);

    return MergeNodes(trees);
}
```

Listing 2: Code for creating the Huffman tree

The implementation here is recursive, with the recursive case passing the new priority queue back to the procedure (to merge again), and the base case being reached when only one tree remains in the priority queue.

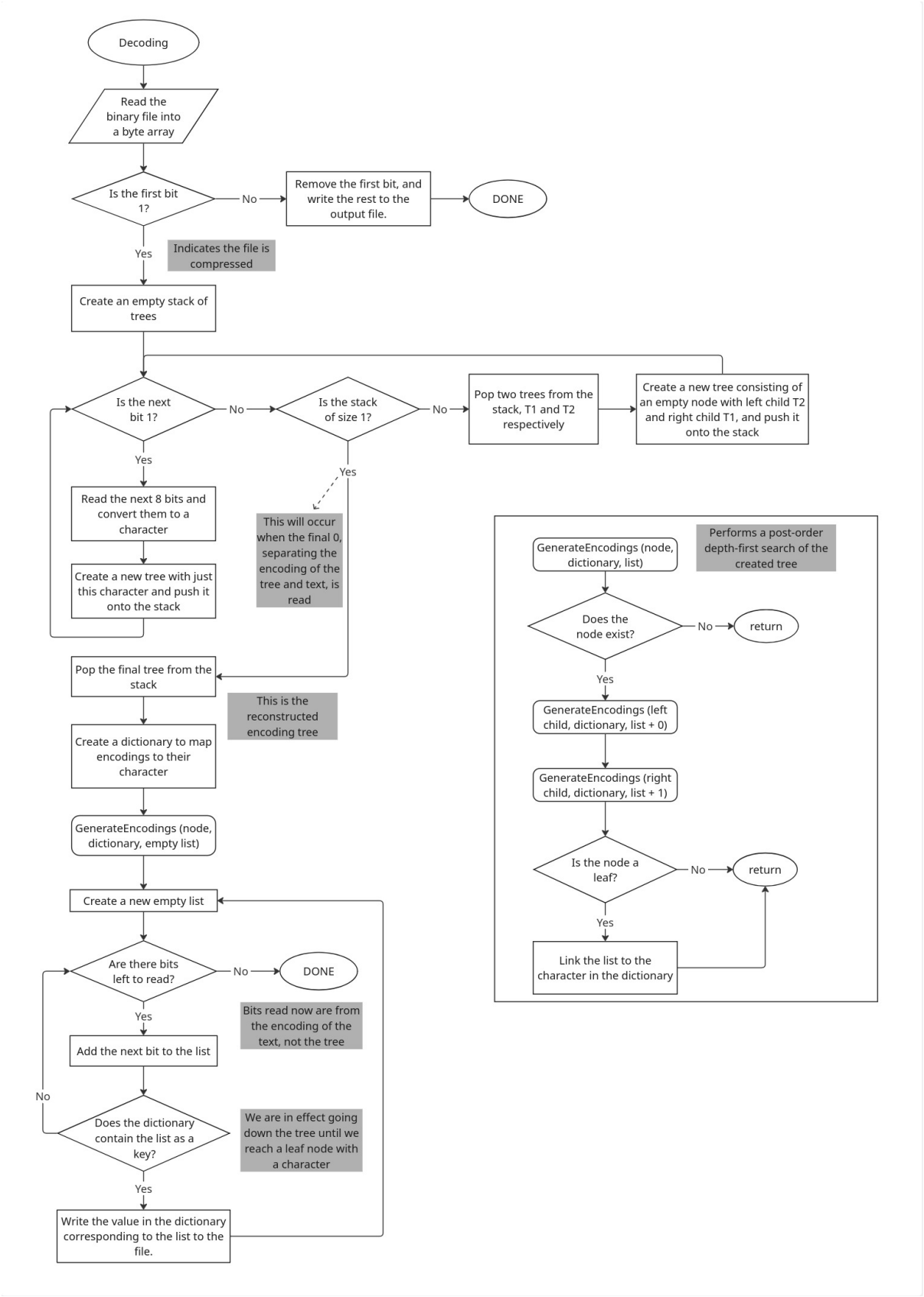

Figure 5: A flowchart depicting my Huffman decoding procedure

In this decoding procedure, the GenerateEncodings procedure recursively calls itself to perform a post-order depth-first search of the Huffman tree to fill a dictionary mapping Huffman encodings to the characters they represent. This works identically to the SearchTree procedure in the encoding procedure, just without writing anything to a file, and with the dictionary direction reversed (in the encoding procedure, characters are linked to their Huffman encoding).

A stack is used to read the Huffman tree representation from the file, and store it in the program as a tree. Figure 6 is a snippet of the main decoding flowchart that relates to this.

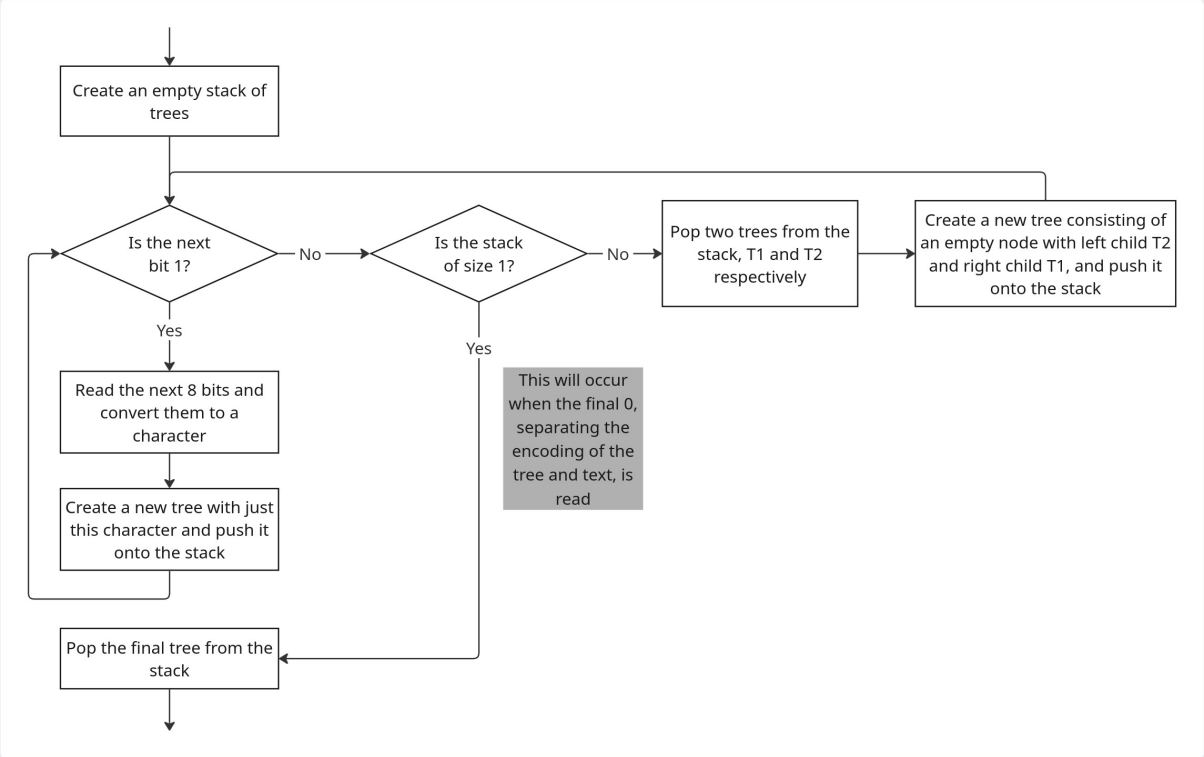

Figure 6: A flowchart depicting the reconstruction of the Huffman tree

The start of the compressed file contains an encoding of the Huffman tree used. This was created, and written, in the encoding procedure, with a post-order depth-first tree search. Here, the tree is reconstructed.

If a 1 bit is read - indicating a character comes next - the next 8 bits are read and converted into a Unicode character, then pushed onto the stack as a new node (which is in effect a tree of size 1).

Otherwise, if a 0 bit is read, one of two options occurs. If the stack is of size 1, this indicates that the zero read represented the end of the tree encoding, and the program pop's the one tree in the stack, and uses that as the Huffman tree to decode the text in the file. Otherwise, the 0 indicates a non-leaf node, so the top two trees are popped from the stack, and made children of a new node which is pushed back onto the stack. The first tree popped becomes the right child.

The code for this algorithm can be found in Listing 3.

```
Stack<HuffmanNode> NodeStack = new Stack<HuffmanNode>();

while (true)
{
    int nextBit = byteReader.Read();

    if (nextBit == 1)
    {
        char? nextChar = byteReader.ReadChar();
        if (nextChar == null) break;

        NodeStack.Push(new HuffmanNode((char)nextChar));
    }
    else
    {
        if (NodeStack.Count() == 1) break;

        HuffmanNode right = NodeStack.Pop();
        HuffmanNode left = NodeStack.Pop();

        NodeStack.Push(new HuffmanNode(left, right));
    }
}

HuffmanNode tree = NodeStack.Pop(); // there should be just one node
   left
```

Listing 3: Code for the reconstructing the Huffman tree

To represent the binary Huffman tree, I made a dedicated HuffmanNode class. Each node is linked to any children it has, so a tree is just represented as its top-node. Leaf nodes also store what character they represent. This class shows polymorphism by having two constructors taking different arguments due to the different information available when making a leaf and non-leaf node.

The pseudocode for this can be found in Listing 4, and the code in Listing 5.

```
CLASS HuffmanNode
        PRIVATE CHARACTER _character
        PRIVATE INTEGER _frequency
        PRIVATE HuffmanNode _left
        PRIVATE HuffmanNode _right

        PRIVATE BOOLEAN _hasCharacter

        PUBLIC CONSTRUCTOR HuffmanNode (CHARACTER char, INTEGER
           frequency)
                _character <- char
                _frequency <- frequency

                _hasCharacter <- TRUE
```

```
        ENDCONSTRUCTOR

        PUBLIC CONSTRUCTOR HuffmanNode (HuffmanNode left, HuffmanNode
           right, INTEGER frequency)
                _left <- left
                _right <- right
                _frequency <- frequency

                _hasCharacter <- FALSE
        ENDCONSTRUCTOR

        PUBLIC SUBROUTINE GetFrequency()
                RETURN _frequency
        ENDSUBROUTINE

        PUBLIC SUBROUTINE HuffmanNode GetLeft()
                RETURN _left
        ENDSUBROUTINE

        PUBLIC SUBROUTINE HuffmanNode GetRight()
                RETURN _right
        ENDSUBROUTINE

        PUBLIC SUBROUTINE bool HasCharacter()
                RETURN _hasCharacter
        ENDSUBROUTINE

        PUBLIC SUBROUTINE char GetCharacter()
                RETURN _character
        ENDSUBROUTINE
ENDCLASS
```

Listing 4: Pseudocode for the HuffmanNode class

```
class HuffmanNode
{
    private char _character;
    private int _frequency;
    private HuffmanNode? _left;
    private HuffmanNode? _right;

    private bool _hasCharacter; // "isLeaf"

    public HuffmanNode(char character, int frequency = 1)
    {
        _character = character;
        _frequency = frequency;

        _hasCharacter = true;
    }

```

```
18      public HuffmanNode(HuffmanNode left, HuffmanNode right, int
            frequency = 1)
19      {
20          _left = left;
21          _right = right;
22          _frequency = frequency;
23
24          _hasCharacter = false;
25      }
26
27      public int GetFrequency() => _frequency;
28
29      public HuffmanNode? GetLeft() => _left;
30      public HuffmanNode? GetRight() => _right;
31
32      public bool HasCharacter() => _hasCharacter;
33      public char GetCharacter() => _character;
34  }
35
36  // NOTE: tree is just a "top-node" - links to left/right
```

Listing 5: Code for the HuffmanNode class

2.1.2 Reading and Writing Bytes

These flowchart implementations above assumed that writing bits to, and reading bits from, files is a trivial task. However, this proved to be quite challenging, since one can only write to, and read from, files one byte at a time in C#. Therefore, I needed to implement additional helper methods for this.

I constructed pseudocode for ByteMaker (Listing 6) and ByteReader (Listing 7) classes, which respectively allow writing to and reading from binary files.

```
1   CLASS ByteMaker
2           PRIVATE INTEGER ARRAY _bits
3           PRIVATE INTEGER _bitIndex
4           PRIVATE INTEGER _trailingZeros
5
6           PRIVATE BYTE LIST _bytes
7
8           PUBLIC CONSTRUCTOR ByteMaker (BOOLEAN isCompressed)
9                   _bitIndex <- 4
10                  IF (isCompressed = TRUE) THEN
11                          _bits <- [1, 0, 0, 0, 0, 0, 0, 0]
12                  ELSE
13                          _bits <- [0, 0, 0, 0, 0, 0, 0, 0]
14                  ENDIF
15          ENDCONSTRUCTOR
16
17          PUBLIC SUBROUTINE Add (INTEGER bit)
```

```
18                  IF (bit <> 1 AND bit <> 0) THEN
19                          RETURN
20                  ENDIF
21
22                  _bits[_bitIndex] <- bit
23                  _bitIndex <- _bitIndex + 1
24
25                  IF (_bitIndex = 8) THEN
26                          updateByte()
27                  ENDIF
28          ENDSUBROUTINE
29
30          PUBLIC SUBROUTINE Add (INTEGER LIST bits)
31                  FOR (INTEGER bit IN bits)
32                          Add(bit)
33                  ENDFOR
34          ENDSUBROUTINE
35
36          PUBLIC SUBROUTINE Add (CHARACTER char)
37                  BYTE charByte <- char TO BYTE
38                  FOR (INTEGER i <- 7 TO 0 STEP -1)
39                          INTEGER bit <- (RIGHTSHIFT charByte i) AND 1
40
41                          Add(bit)
42                  ENDFOR
43          ENDSUBROUTINE
44
45          PUBLIC SUBROUTINE Export()
46                  updateByte()
47
48                  BYTE trailingZerosByte <- _trailingZeros TO BYTE
49                  _bytes[0] <- _bytes[0] OR (LEFTSHIFT trailingZerosByte
                        4)
50
51                  RETURN (_bytes TO ARRAY)
52          ENDSUBROUTINE
53
54          PRIVATE SUBROUTINE updateByte()
55                  BYTE newByte
56                  FOR (INTEGER i <- 0 TO 7 STEP 1)
57                          BYTE mergeByte <- (LEFTSHIFT _bits[i] 7-i) TO
                               BYTE
58                          newByte <- newByte OR mergeByte
59                  ENDFOR
60                  ADD newByte TO _bytes
61
62                  _bits <- [0, 0, 0, 0, 0, 0, 0, 0]
63                  _trailingZeros <- 8 - _bitIndex
64                  _bitIndex <- 0
65          ENDSUBROUTINE
66  ENDCLASS
```

Listing 6: Pseudocode for the ByteMaker class

The ByteMaker provides methods to write single bits (as integers 0/1), lists of bits, and characters to a private list of bytes, which it can "export" upon completion, returning a byte array that can be written to the binary file. The class holds an 8 long array of integers to represent the current byte, and this is added to the byte array when it is filled (when the index used to write to it reaches 8). This is done by converting it into a byte using bitwise operations (shifting each bit to its position, then performing an OR operation with the byte being constructed).

When all desired bits have been written to the file, it is likely that this is not at the end of a byte, and so the remaining bits are left as 0, since you can only write to files in bytes. However, the number of such bits need to be stored in some form, so it is clear which bits to ignore when decoding. Therefore, bits 1-3 of the first byte are kept clear for writing a 3-bit integer between 0-7 (the possible number of trailing bits) at the end. Note that bit 0 is used to indicate whether the file is being compressed in the first place. This is why the index begins at 4 rather than 0 for the first bit.

```
1  CLASS ByteReader
2          PRIVATE BYTE LIST _bytes
3          PRIVATE INTEGER _bitIndex
4          PRIVATE INTEGER _trailingNum
5
6          PUBLIC CONSTRUCTOR ByteReader (BYTE ARRAY bytes)
7                  _bytes <- bytes TO LIST
8                  _bitIndex <- 0
9
10                 INTEGER firstBit <- Read()
11                 INTEGER a <- Read()
12                 INTEGER b <- Read()
13                 INTEGER c <- Read()
14                 _trailingNum <- 4*a + 2*b + c
15
16                 IF (firstBit = 0) THEN
17                         RETURN FALSE
18                 ELSE
19                         RETURN TRUE
20                 ENDIF
21         ENDCONSTRUCTOR
22
23         PUBLIC SUBROUTINE Read()
24                 IF (LENGTH _bytes = 0) THEN
25                         RETURN -1
26                 ENDIF
27                 IF (LENGTH _bytes = 1 AND _bitIndex >= 8 -
                      _trailingNum) THEN
28                         RETURN -1
29                 ENDIF
30
```

```
                INTEGER bit <- (RIGHTSHIFT _bytes[0] (7 - _bitIndex))
                    AND 1
                _bitIndex <- _bitIndex + 1

                RETURN bit
        ENDSUBROUTINE

        PUBLIC SUBROUTINE ReadChar()
                BYTE nextByte <- 0

                FOR (INTEGER i <- 0 TO 7 STEP 1)
                        INTEGER bit <- Read()
                        IF (bit = -1) THEN
                                RETURN ''
                        ELSE IF (bit = 1) THEN
                                nextByte <- nextByte OR (LEFTSHIFT 1
                                    (7-i))
                        ENDIF
                ENDFOR

                RETURN (nextByte TO CHARACTER)
        ENDSUBROUTINE

        PRIVATE SUBROUTINE Update()
                IF (_bitIndex <> 8) THEN
                        RETURN
                ENDIF

                REMOVE INDEX 0 FROM _bytes
                _bitIndex <- 0
        ENDSUBROUTINE
ENDCLASS
```

Listing 7: Pseudocode for the ByteReader class

This ByteReader class acts as the opposite of the ByteMaker: it takes in a byte array when being instantiated, and provides methods for reading the next bit and character (8 bits). The constructor returns a boolean value indicating whether the first bit is 0 or 1, and so whether compression was used on the file. It also uses the next 3 bits to determine the number of trailing 0s at the end of the file, so they can be ignored when reading. This is indicated by the Read method returning -1 once those characters have been reached.

The Read method uses bitwise operations to get the bit at a certain index from the byte at index 0 of the list. Index 0 is always used as the Update method removes bytes that have already been fully read from the start of the list. The right shift operation places the desired bit at the end of the byte, and performing an AND with 1 returns the value of the last bit of the right-shifted byte, as desired.

The code for the ByteMaker and ByteReader classes can be found in Listings 8 and 9 respectively.

```
class ByteMaker
```

```
{
    private int[] _bits;
    private int _bitIndex;
    private int _trailingZeros;

    private List<byte> _bytes;

    public ByteMaker(bool isCompressed)
    {
        _bits = [isCompressed ? 1 : 0, 0, 0, 0, 0, 0, 0, 0]; // length
            8
        // first bit indicates compressed, next 3 bits for trailing
            zeros
        _bitIndex = 4;
        _trailingZeros = 0;

        _bytes = new List<byte>();
    }

    public void Add(int bit)
    {
        if (bit != 1 && bit != 0) return;

        _bits[_bitIndex++] = bit;

        if (_bitIndex == 8) updateByte();
    }

    public void Add(List<int> bits)
    {
        foreach (int bit in bits) Add(bit);
    }

    public void Add(char character)
    {
        byte characterByte = (byte)character;

        for (int i = 7; i >= 0; i--)
        {
            // right-shift, then AND with 1 to get "last" bit
            int bit = (characterByte >> i) & 1;

            Add(bit);
        }
    }

    public byte[] Export()
    {
        updateByte();

        // push to bits 1-4, OR since already 0
```

```
        _bytes[0] |= (byte)(_trailingZeros << 4);

        return _bytes.ToArray();
    }

    private void updateByte()
    {
        byte newByte = new byte();

        for (int i = 0; i < 8; i++)
        {
            // left-shift, then OR with existing byte
            newByte |= (byte)(_bits[i] << (7-i));
        }

        _bytes.Add(newByte);

        _bits = [0, 0, 0, 0, 0, 0, 0, 0]; // length 8
        _trailingZeros = 8 - _bitIndex;
        _bitIndex = 0;
    }
}
```

Listing 8: Code for the ByteMaker class

```
class ByteReader
{
    private List<byte> _bytes;
    private int _bitIndex;
    private int _trailingNum;

    public ByteReader(byte[] bytes, out bool compressed)
    {
        _bytes = bytes.ToList();
        _bitIndex = 0;

        int firstBit = Read();

        if (firstBit == 0) compressed = false;
        else compressed = true;

        int a = Read();
        int b = Read();
        int c = Read();

        _trailingNum = a*4 + b*2 + c;
    }

    public int Read()
    {
        if (_bytes.Count() == 0) return -1;
```

```
27
28          // don't read padding zeros
29          if (_bytes.Count() == 1 && _bitIndex >= 8 - _trailingNum)
                return -1;
30
31          int bit = (_bytes[0] >> (7 - _bitIndex++)) & 1;
32
33          Update();
34
35          return bit;
36      }
37
38      public char? ReadChar()
39      {
40          byte nextByte = 0;
41
42          for (int i = 0; i < 8; i++)
43          {
44              int bit = Read();
45              if (bit == -1) return null;
46
47              // shift 1 to its position and OR with exisiting byte
48              if (bit == 1) nextByte |= (byte)(1 << (7-i));
49          }
50
51          return (char)nextByte;
52      }
53
54      private void Update()
55      {
56          if (_bitIndex != 8) return;
57
58          _bytes.RemoveAt(0);
59          _bitIndex = 0;
60      }
61  }
```

Listing 9: Code for the ByteReader class

2.1.3 UML Class Diagram

Figure 7 shows a UML class diagram depicting my design for this component of the program. No inheritance, composition, or aggregation is used, but the HuffmanNode constructor and the ByteMaker Add method show polymorphism by having overloads.

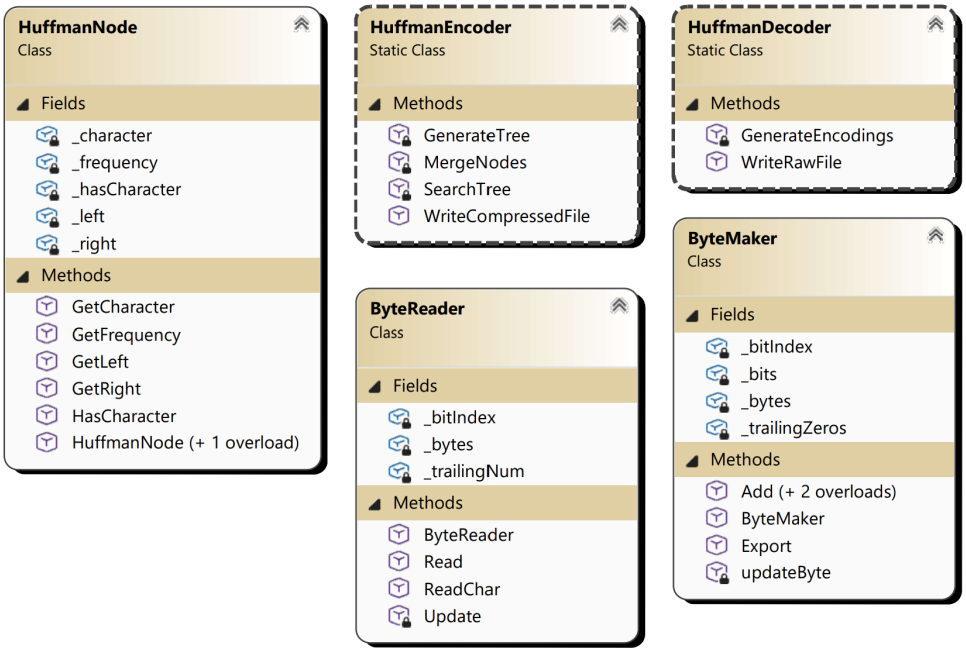

Figure 7: A UML class diagram for the compression program

2.2 RSA Key Exchange

In my Analysis (Section 1.7.1), I decided that I will use the RSA algorithm for key exchange. This asymmetric encryption algorithm will be used to share a symmetric key between the client and server, to be used for any further communications between the devices.

From the Analysis, the following procedure can be followed to generate RSA keys:

1. The receiver generates two large primes, p, q, then computes their product, $n = pq$.
2. Euler's Totient function, $\phi(pq) = (p-1)(q-1)$, is applied to n.
3. An encryption exponent, e, is selected such that $1 < e < \phi(n)$, and e is relatively prime with $\phi(n)$. Note that $a, b \in \mathbb{Z}^+$ are relatively prime $\iff \nexists\, x \in \mathbb{Z} : x > 1,\ x \mid a$, and $x \mid b$.
4. A decryption exponent, d, is calculated such that $de = 1 \pmod{\phi(n)}$. The extended Euclidian algorithm provides a method for this.
5. The public key is formed from (n, e) and the private key (n, d). The receiver sends their public key only to the sender.
6. The encrypted ciphertext, C, is calculated through $C = M^e \pmod{n}$, where M is some numerical representation of the message.
7. The ciphertext is decrypted into the plaintext message, M, by the calculation $M = C^d \pmod{n}$.

The first prerequisite to implementing the RSA algorithm is a method to generate very large (512 bit) prime numbers. Since it is intractable to generate random very large prime numbers that are guaranteed to be prime, the Miller-Rabin primality test is very often used. This is an algorithm that tests whether a large prime number is probably prime. Wikipedia provides a description of this algorithm [5].

For each base tested, if this algorithm labels an integer composite, it must be composite, but it has a $\frac{1}{4}$ probability of incorrectly labelling the integer prime. However, this means that running the algorithm n times (with n bases) reduces this probability of incorrectly labelling a composite number prime every time (since if it ever labels it composite then it must be composite) to 4^{-n}. Running this 40 times, the probability of it giving a wrong result is 4^{-40}, which is exceedingly unlikely.

Figure 8 displays a flowchart demonstrating the steps this algorithm takes.

Listing 10 contains the code for this algorithm.

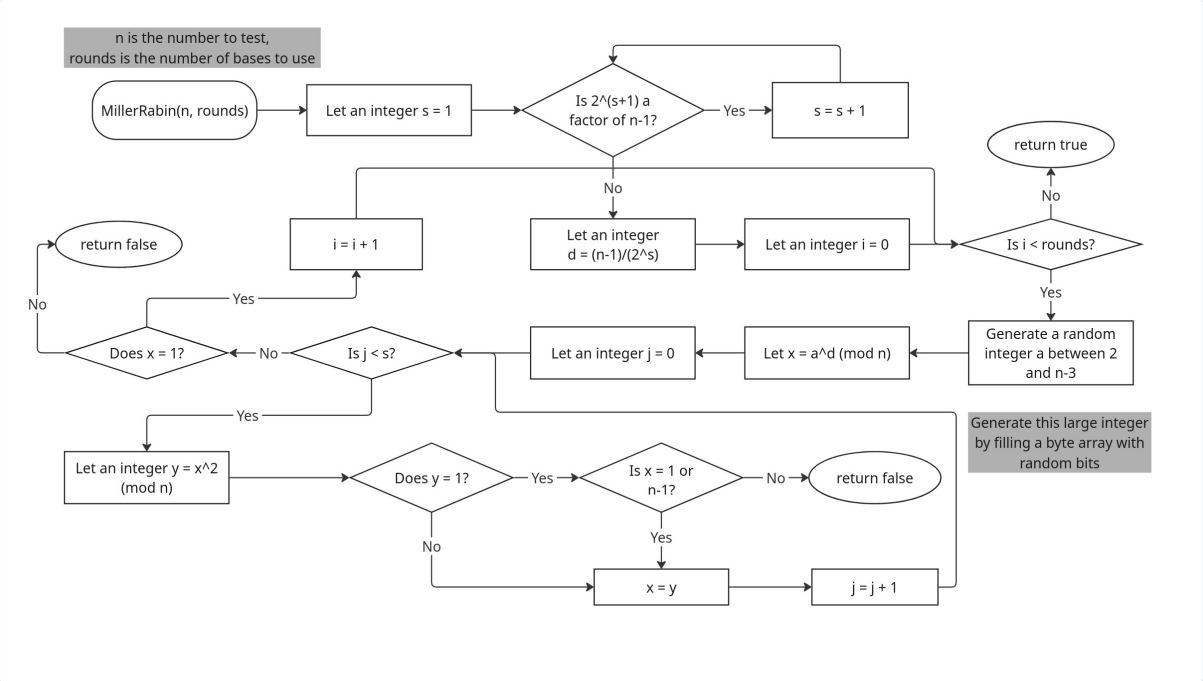

Figure 8: A flowchart for the Miller-Rabin algorithm

```
private static bool MillerRabin(BigInteger n, int rounds)
{
    int s = 1;
    while ((n-1) % (BigInteger)Math.Pow(2, s+1) == 0) s++;

    BigInteger d = (n-1)/(BigInteger)(Math.Pow(2,s));

    for (int i = 0; i < rounds; i++)
    {
        BigInteger a = GenerateInRange(2, n-3);

        BigInteger x = BigInteger.ModPow(a, d, n);

        for (int j = 0; j < s; j++)
        {
            BigInteger y = BigInteger.ModPow(x, 2, n);

            if (y == 1 && x != 1 && x != n-1) return false;

            x = y;
        }

        if (x != 1) return false;
    }

    return true;
}
```

Listing 10: Code for the MillerRabin algorithm

This is used to generate large primes by continually testing randomly generated primes until one passes the test. Large primes (512 bit in this case) are generated by filling a byte array (64 long in this case) with random bits, ensuring the most significant bit is 1, so the prime is actually large enough, and the least significant bit is also 1, so the number is not a multiple of two (since they are clearly not prime). A check is also performed to ensure that for the composite n formed by $n = pq$, where p, q are primes being generated here, $\phi(n)$ is relatively prime with the encryption exponent e, which is chosen as 65537 in this case, for its favourable properties.

A flowchart for this procedure can be found in Figure 9, and code for it in Listing 11.

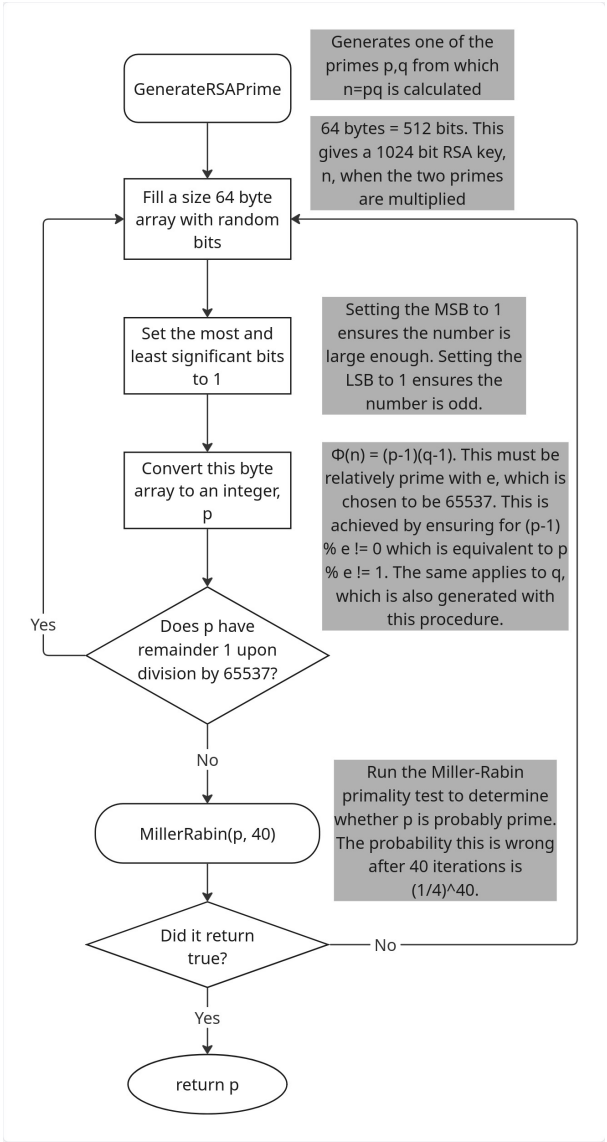

Figure 9: A flowchart for the procedure to generate an RSA prime

```
1 private static BigInteger GenerateRSAPrime(BigInteger e) // e must be
    prime in this implementation!
2 {
```

```
    while (true)
    {
        BigInteger bigInt;

        do bigInt = GenerateOdd512BitInt();
        while (bigInt % e == 1); // ensures e will be relatively prime
            with phi(n), as e used is prime

        if (MillerRabin(bigInt, 40)) return bigInt;
    }
}

private static BigInteger GenerateOdd512BitInt()
{
    byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
    RandomNumberGenerator.Fill(randomInt);

    randomInt[0] |= (byte)(128); // sets MSB to 1, so num is 512 bits

    randomInt[63] |= (byte)(1); // sets LSB to 1 - odd

    return new BigInteger(randomInt, true, true); // isUnsigned,
        isBigEndian
}
```

Listing 11: Code for the procedure to generate an RSA prime

The final major algorithm needed in the implementation of RSA is the Extended Euclidean Algorithm, which is used to generate the decryption exponent, d, such that $de = 1 \pmod{\phi(n)}$. Wikipedia provides a mathematical description of this algorithm [6]. Figure 10 provides a flowchart representing the algorithm for this, and Listing 12 shows the code for this procedure.

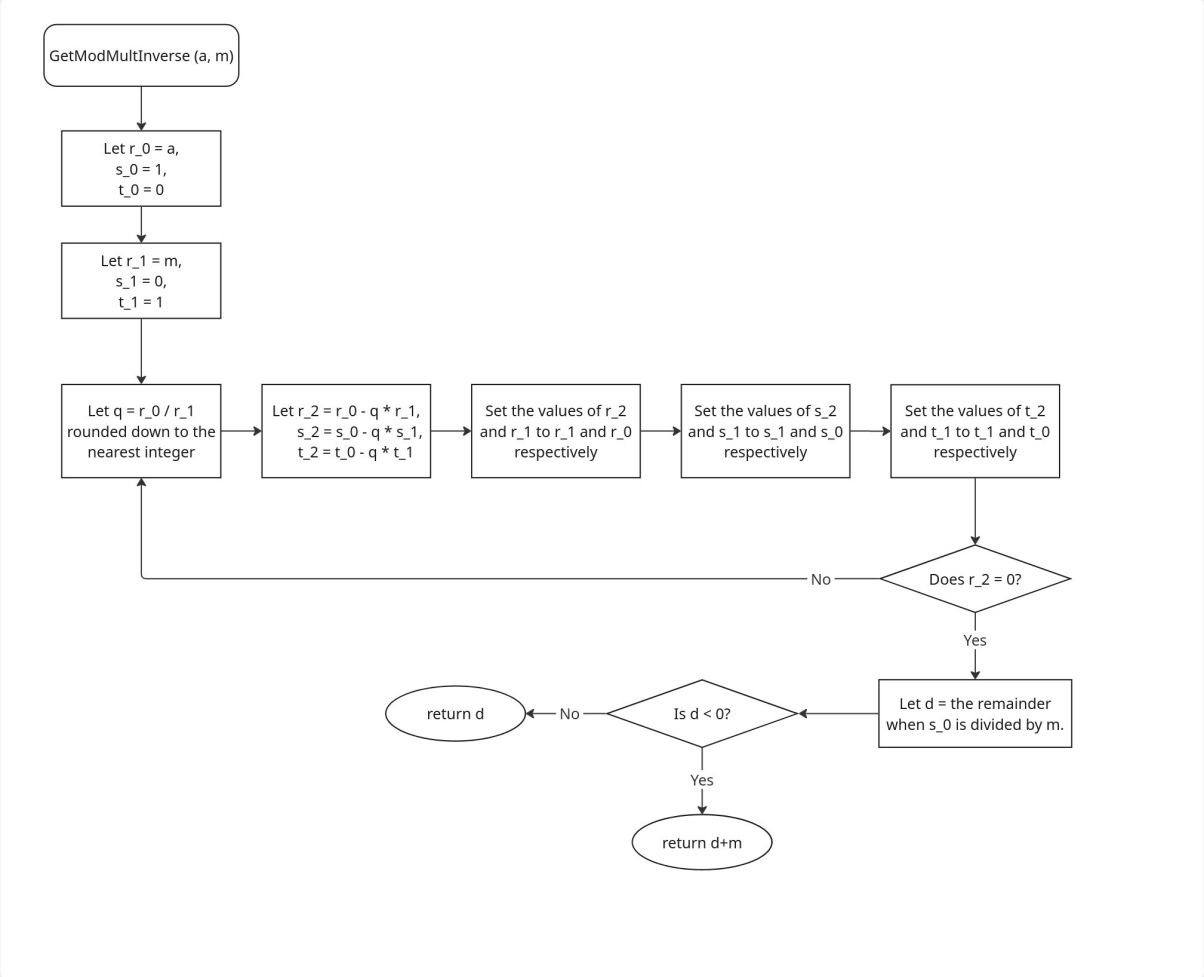

Figure 10: A flowchart for the Extended Euclidean Algorithm

```
1  private static BigInteger GetModMultInverse(BigInteger a, BigInteger
       m) // uses the extended euclidian algorithm to find x s.t. ax = 1
       (mod m)
2  {
3      BigInteger r_0 = a;
4      BigInteger s_0 = 1;
5      BigInteger t_0 = 0;
6
7      BigInteger r_1 = m;
8      BigInteger s_1 = 0;
9      BigInteger t_1 = 1;
10
11     BigInteger r_2, s_2, t_2;
12
13     do
14     {
15         BigInteger q = r_0 / r_1;
16
17         r_2 = r_0 - q * r_1;
18         s_2 = s_0 - q * s_1;
```

```
        t_2 = t_0 - q * t_1;

        (r_0, r_1) = (r_1, r_2);
        (s_0, s_1) = (s_1, s_2);
        (t_0, t_1) = (t_1, t_2);

    } while (r_2 != 0);

    BigInteger d = s_0 % m;
    if (d < 0) d += m;

    return d;
}
```

Listing 12: Code for the Extended Euclidean Algorithm

All that is left now is to put this all together, and generate the public and private RSA keys. The public key consists of (n, e), and the private key (n, d), so a procedure is needed to generate the tuple (n, e, d). Figure 11 provides a flowchart for this, and my implementation can be seen in Listing 13.

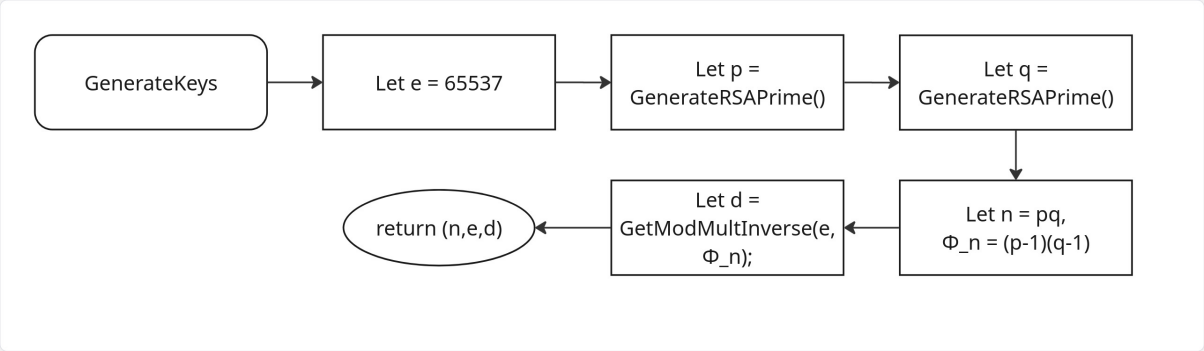

Figure 11: A flowchart for the procedure to generate RSA keys

```
public static Tuple<BigInteger, BigInteger, BigInteger> GenerateKeys()
{
    BigInteger e = 65537;

    BigInteger p = RSA.GenerateRSAPrime(e);
    BigInteger q = RSA.GenerateRSAPrime(e);

    BigInteger n = p*q;
    BigInteger phin = (p-1)*(q-1);

    BigInteger d = RSA.GetModMultInverse(e, phin);

    return new Tuple<BigInteger, BigInteger, BigInteger>(n,e,d);
}
```

Listing 13: Code for the procedure to generate RSA keys

2.3 Verifying the integrity of communications

To verify the integrity of the files transmitted, as described in the Analysis (Section 1.7.3), I will compute a hash of the data being sent, and transmit the hash of the message before the rest of the message. The recipient can then compute the hash of the message itself, and compare that to the hash it received. If they match, then they can be confident that the message was not corrupted in transmission.

As decided in the Analysis, I will use the popular SHA-256 algorithm for this, which generates a fixed size 256-bit hash. Listing 14 contains pseudocode for the procedures to generate a hash, and to verify one.

```
PUBLIC SUBROUTINE Hash (BYTE ARRAY messageBytes)
        BYTE ARRAY hashBytes

        hashBytes <- GET HASH OF messageBytes

        RETURN hashBytes
ENDSUBROUTINE

PUBLIC SUBROUTINE VerifyHash (BYTE ARRAY messageBytes, BYTE ARRAY
   hashBytes)
        BYTE ARRAY messageHash <- Hash(messageBytes)

        FOR (INTEGER i <- 0 TO (LENGTH OF messageHash - 1) STEP 1)
                IF (messageHash[i] <> hashBytes[i]) THEN
                        RETURN FALSE
                ENDIF
        ENDFOR

        RETURN TRUE
ENDSUBROUTINE
```

Listing 14: Pseudocode for the hashing procedures

The verification subroutine checks each byte in the byte arrays representing the hash computed and hash received, and returns false if any do not match exactly. Code for this can be found in Listing 15.

```
using System.Security.Cryptography;

static class Hashing
{
    public static byte[] Hash(byte[] messageBytes)
    {
        byte[] hashBytes;

        using (SHA256 HashMaker = SHA256.Create())
        {
            hashBytes = HashMaker.ComputeHash(messageBytes);
        }
```

```
13
14          return hashBytes;
15      }
16
17      public static bool VerifyHash(byte[] messageBytes, byte[] hash)
18      {
19          byte[] originalHashed = Hash(messageBytes);
20
21          if (originalHashed.Length != hash.Length) return false;
22
23          for (int i = 0; i < hash.Length; i++)
24          {
25              if (originalHashed[i] != hash[i]) return false;
26          }
27
28          return true;
29      }
30  }
```

Listing 15: Code for the hashing procedures

2.4 Transmission procedures

In order to send messages through a network stream, at the simplest level, there must be a method for the recipient to know how long the data being sent is. I can achieve this by encoding the number of bytes to expect in the first 4 bytes of each transmission. As such, the receiver knows to read the first 4 bytes (which will be an integer n), read the next n bytes in whatever format it is expecting, and then repeat this procedure for the rest of the stream, knowing that it is a new transmission.

This requires some serialisation and "de-serialisation" subroutines to handle adding the length of the message to the start of it, and reading exactly as many bytes as are indicated. Listing 16 provides pseudocode for these methods, and Listing 17 provides the code for it.

```
1   CLASS Transmission
2           PUBLIC SUBROUTINE Serialise (BYTE ARRAY bytes)
3                   BYTE ARRAY[4] bytesLengthBytes <- (LENGTH OF bytes) TO
                        BYTE ARRAY
4
5                   BYTE ARRAY data <- bytesLengthBytes + bytes
6
7                   RETURN data
8           ENDSUBROUTINE
9
10          PUBLIC SUBROUTINE Serialise (BIG INTEGER bigInt)
11                  BYTE ARRAY bigIntBytes <- bigInt TO BYTE ARRAY
12
13                  RETURN Serialise(bigIntBytes)
14          ENDSUBROUTINE
```

```
15
16          PUBLIC SUBROUTINE Serialise (STRING data)
17                  RETURN Serialise(data TO BYTE ARRAY)
18          ENDSUBROUTINE
19
20          PUBLIC SUBROUTINE ReadByteArray (NETWORK STREAM stream)
21                  INTEGER bytesLength <- ReadBytes(stream, 4) TO INTEGER
22
23                  RETURN ReadBytes(stream, bytesLength)
24          ENDSUBROUTINE
25
26          PUBLIC SUBROUTINE ReadBigInt (NETWORK STREAM stream)
27                  RETURN ReadByteArray(stream) TO BIG INTEGER
28          ENDSUBROUTINE
29
30          public static byte[] ReadBytes(Stream stream, int length)
31                  BYTE ARRAY[length] bytes
32
33                  bytes <- READ length FROM stream
34
35                  RETURN bytes
36          ENDSUBROUTINE
37  ENDCLASS
```

Listing 16: Pseudocode for the serialisation procedures

```
1   using System.Numerics;
2   using System.Net.Sockets;
3
4   static class Transmission
5   {
6       public static byte[] Serialise(byte[] bytes)
7       {
8           byte[] bytesLengthBytes = BitConverter.GetBytes(bytes.Length);
                // length 4 always
9
10          byte[] data =
                bytesLengthBytes.ToList().Concat(bytes.ToList()).ToArray();
11
12          return data;
13      }
14
15      public static byte[] Serialise(BigInteger bigInt)
16      {
17          byte[] bigIntBytes = bigInt.ToByteArray(true, true); //
                isUnsigned, isBigEndian
18
19          return Serialise(bigIntBytes);
20      }
21
22      public static byte[] Serialise(string data) =>
```

```
        Serialise(System.Text.Encoding.ASCII.GetBytes(data));
23
24      public static BigInteger ReadBigInt(NetworkStream stream) => new
            BigInteger(ReadByteArray(stream), true, true);
25
26      public static byte[] ReadByteArray(Stream stream)
27      {
28          int bytesLength = BitConverter.ToInt32(ReadBytes(stream, 4));
29
30          return ReadBytes(stream, bytesLength);
31      }
32
33      public static byte[] ReadBytes(Stream stream, int length)
34      {
35          byte[] bytes = new byte[length];
36
37          int read = 0;
38          while (read < length) read += stream.Read(bytes, read, length
                - read);
39
40          return bytes;
41      }
42  }
```

Listing 17: Code for the serialisation procedures

Having established an encrypted AES stream (using the CryptoStream class in C#), some helper methods are needed to send and receive various data types. I designed FileSender and FileReciever classes for this purpose. These both inherit from an abstract Transferer class, which stores the common CryptoStream attribute, and a common constructor for the two derived classes. Using classes, rather than just procedures, is beneficial here since, as well as grouping together the related subroutines cleanly, the CryptoStream can be passed in just once upon instantiation of an object. It is important that this is not re-created, since that would interrupt the communication stream. Pseudocode for these classes can be found in Listing 18, and the code in Listing 19.

```
1   ABSTRACT CLASS Transferer
2           PROTECTED CRYPTO STREAM _cryptoStream
3
4       PUBLIC CONSTRUCTOR Transferer (NETWORK STREAM stream, AES
            ENCRYPTOR aes, CRYPTO STREAM MODE mode)
5                   SET aes PADDING MODE TO ZEROS
6
7                   IF (mode = WRITE) THEN
8                           _cryptoStream <- NEW CRYPTO STREAM(stream, aes
                                ENCRYPTOR, mode, LEAVE OPEN)
9                   ELSE
10                          _cryptoStream <- NEW CRYPTO STREAM(stream, aes
                                DECRYPTOR, mode, LEAVE OPEN)
11                  ENDIF
12          ENDCONSTRUCTOR
```

```
13 ENDCLASS
14
15 CLASS FileSender FROM Transferer
16     PUBLIC CONSTRUCTOR FileSender (NETWORK STREAM stream, AES
           ENCYRPTOR aes)
17                 CALL BASE(stream, aes, WRITE)
18         ENDCONSTRUCTOR
19
20         PUBLIC SUBROUTINE SendFile (BYTE ARRAY fileBytes, STRING
               fileName)
21                 BYTE ARRAY hashBytes <- Hash(fileBytes)
22
23                 WRITE Serialise(hashBytes) TO _cryptoStream
24
25                 WRITE Serialise(fileName) TO _cryptoStream
26
27                 SendData(fileBytes)
28         ENDSUBROUTINE
29
30         PUBLIC SUBROUTINE SendData (STRING data)
31                 BYTE ARRAY bytes <- data TO BYTE ARRAY
32
33                 BYTE ARRAY hashBytes <- HASH(bytes)
34
35                 WRITE Serialise(hashBytes) TO _cryptoStream
36
37                 SendData(bytes)
38         ENDSUBROUTINE
39
40         PUBLIC SUBROUTINE SendInt (INTEGER data)
41                 BYTE ARRAY bytes[4] <- data TO BYTE ARRAY
42
43                 SendData(bytes)
44         ENDSUBROUTINE
45
46         PRIVATE SUBROUTINE SendData (BYTE ARRAY bytes)
47                 WRITE Serialise(bytes) TO _cryptoStream
48                 FLUSH _cryptoStream
49         ENDSUBROUTINE
50
51         PUBLIC SUBROUTINE Finish()
52                 FLUSH FINAL BLOCK OF _cryptoStream
53         ENDSUBROUTINE
54 ENDCLASS
55
56 CLASS FileReceiver FROM Transferer
57         PUBLIC CONSTRUCTOR FileReceiver (NETWORK STREAM stream, AES
               aes)
58                 CALL BASE(stream, aes, READ)
59         ENDCONSTRUCTOR
60
```

```
        PUBLIC SUBROUTINE GetFile (STRING location, BOOLEAN decompress)
                BYTE ARRAY hashBytes <- ReadByteArray(_cryptoStream)
                BYTE ARRAY nameBytes <- ReadByteArray(_cryptoStream)
                BYTE ARRAY fileBytes <- ReadByteArray(_cryptoStream)

                IF (VerifyHash(fileBytes, hashBytes) = TRUE) THEN
                        STRING name <- nameBytes TO STRING
                        STRING filePath <- location + name

                        CREATE DIRECTORY AT filePath

                        IF (decompress = TRUE AND LAST ELEMENT OF
                           (SPLIT name BY '.') = "huff") THEN
                                STRING outputPath <-
                                   SUBSTRING(filePath, 0, LENGTH OF
                                   filePath - 5)

                                HuffmanDecoder.WriteRawFile(fileBytes,
                                   outputPath)
                        ELSE
                                WRITE fileBytes TO FILE filePath
                        ENDIF
                ELSE
                        OUTPUT "Corrupted transmission!"
                ENDIF
        ENDSUBROUTINE

        PUBLIC SUBROUTINE GetString()
                BYTE ARRAY hashBytes <- ReadByteArray(_cryptoStream)
                BYTE ARRAY stringBytes <- ReadByteArray(_cryptoStream)

                IF (VerifyHash(stringBytes, hashBytes) = TRUE) THEN
                        RETURN stringBytes TO STRING
                ENDIF

                RETURN ""
        ENDSUBROUTINE

        PUBLIC SUBROUTINE GetInt()
                BYTE ARRAY intBytes <- ReadByteArray(_cryptoStream)

                RETURN intBytes TO INTEGER
        ENDSUBROUTINE
ENDCLASS
```

Listing 18: Pseudocode for the transfer classes

```
using System.Net.Sockets;
using System.Security.Cryptography;

abstract class Transferer
```

```
{
    protected CryptoStream _cryptoStream;

    public Transferer(NetworkStream stream, Aes aes, CryptoStreamMode
        mode)
    {
        aes.Padding = PaddingMode.Zeros;

        _cryptoStream = new CryptoStream(stream, mode ==
            CryptoStreamMode.Write ? aes.CreateEncryptor() :
            aes.CreateDecryptor(), mode, leaveOpen: true);
    }
}

class FileSender : Transferer
{
    public FileSender(NetworkStream stream, Aes aes) : base(stream,
        aes, CryptoStreamMode.Write) { }

    public void SendFile(byte[] fileBytes, string fileName)
    {
        _cryptoStream.Write(
                Transmission.Serialise(Hashing.Hash(fileBytes))
            );

        _cryptoStream.Write(Transmission.Serialise(fileName));

        SendData(fileBytes);
    }

    public void SendData(string data)
    {
        byte[] bytes = System.Text.Encoding.ASCII.GetBytes(data);

        _cryptoStream.Write(
                Transmission.Serialise(Hashing.Hash(bytes))
            );

        SendData(bytes);
    }

    public void SendInt(int data)
    {
        byte[] bytes = BitConverter.GetBytes(data); // 4 bytes

        SendData(bytes);
    }

    private void SendData(byte[] bytes)
    {
        _cryptoStream.Write(Transmission.Serialise(bytes));
```

```
52          _cryptoStream.Flush();
53      }
54
55      public void Finish() => _cryptoStream.FlushFinalBlock();
56  }
57
58  class FileReceiver : Transferer
59  {
60      public FileReceiver(NetworkStream stream, Aes aes) : base(stream,
            aes, CryptoStreamMode.Read) { }
61
62      public void GetFile(string location, bool decompress = false)
63      {
64          byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
                // 32 byte hash read
65          byte[] nameBytes = Transmission.ReadByteArray(_cryptoStream);
                // read file name
66          byte[] fileBytes = Transmission.ReadByteArray(_cryptoStream);
67
68          if (Hashing.VerifyHash(fileBytes, hashBytes))
69          {
70              string name =
                    System.Text.Encoding.ASCII.GetString(nameBytes);
71
72              string filePath = location + name;
73
74              Directory.CreateDirectory(Path.GetDirectoryName(filePath)
                    ?? "");
75
76              if (decompress && name.Split('.').LastOrDefault() ==
                    "huff")
77              {
78                  HuffmanDecoder.WriteRawFile(fileBytes,
                        filePath.Substring(0, filePath.Length - 5));
79              }
80              else File.WriteAllBytes(filePath, fileBytes);
81          }
82          else Console.WriteLine("Corrupted transmission!");
83      }
84
85      public string GetString()
86      {
87          byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
88          byte[] stringBytes = Transmission.ReadByteArray(_cryptoStream);
89
90          if (Hashing.VerifyHash(stringBytes, hashBytes))
91          {
92              return System.Text.Encoding.ASCII.GetString(stringBytes);
93          }
94
95          return "";
```

```
96      }
97
98      public int GetInt()
99      {
100         byte[] intBytes = Transmission.ReadByteArray(_cryptoStream);
101
102         return BitConverter.ToInt32(intBytes);
103     }
104 }
```

Listing 19: Code for the transfer classes

2.4.1 UML Class Diagram

A UML class diagram for the above classes can be seen in Figure 12, demonstrating the inheritance from the base Transferer class.

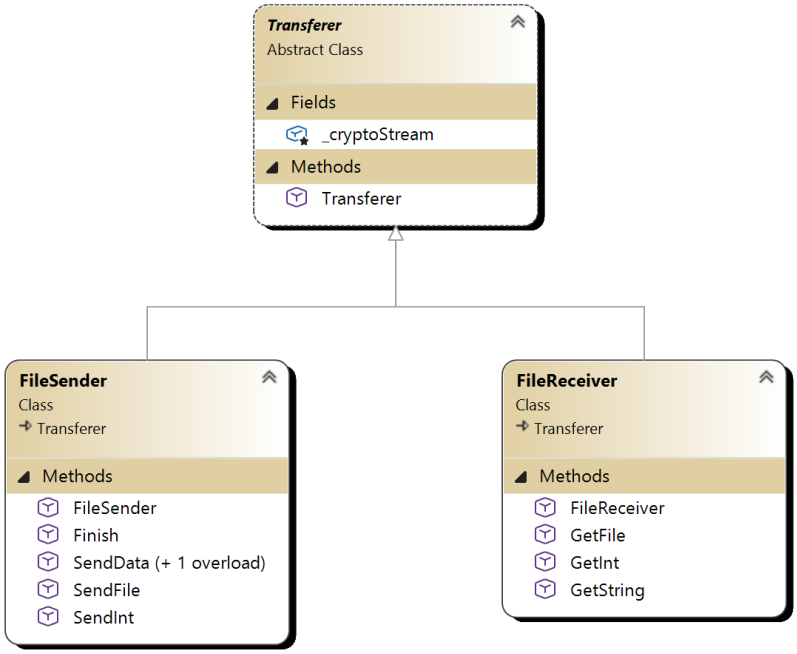

Figure 12: A UML class diagram for the Transferer, FileSender, and FileReceiver classes

2.5 Full UML Class Diagram

Figure 13 shows a UML class diagram for my design of the whole program. Polymorphism is often shown through overloaded methods, and there are two derived classes. However, often, classes are static and simply used for the grouping of related methods.

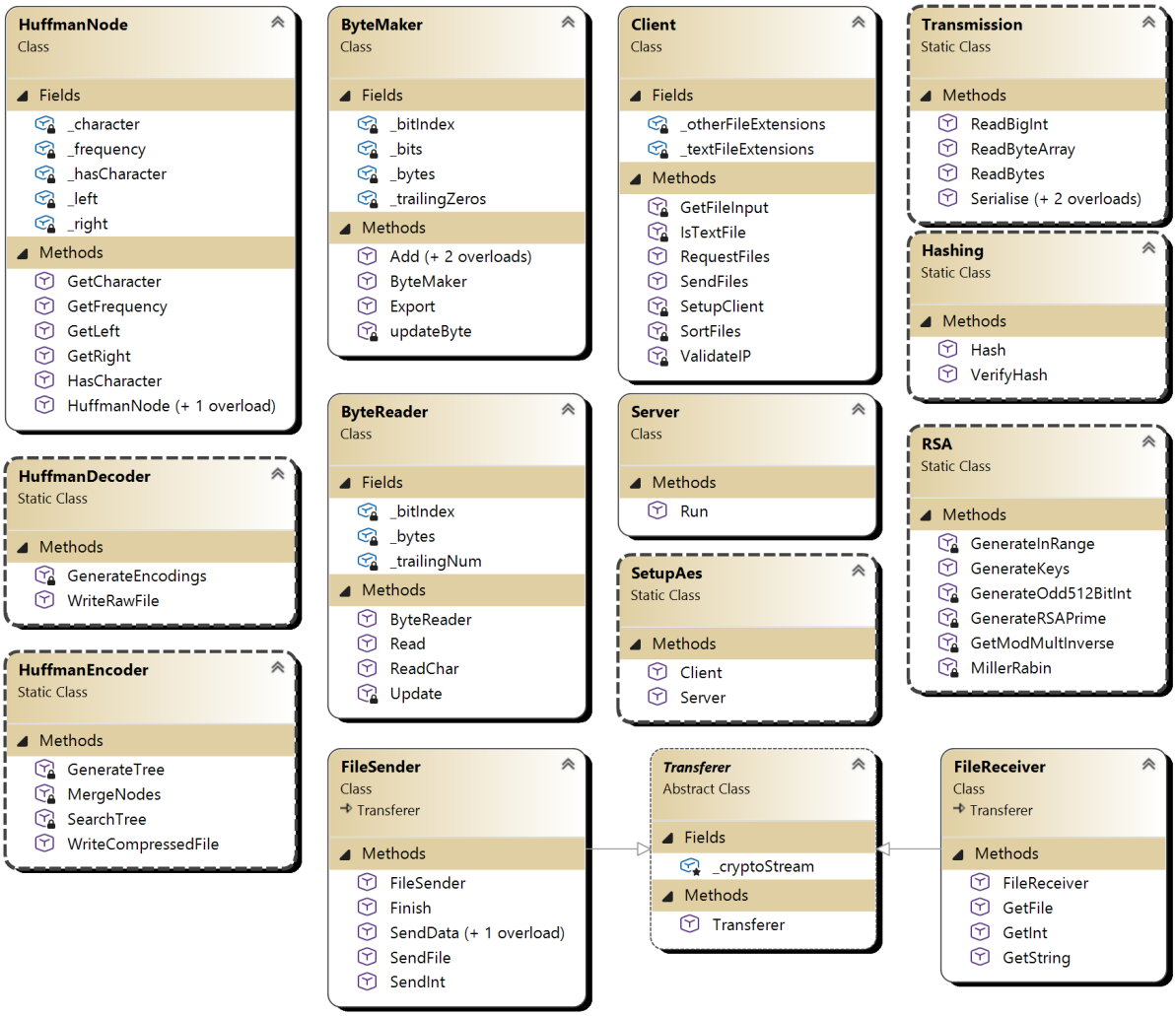

Figure 13: A UML class diagram for the whole program

3 Technical Solution

3.1 Main Program

```
Client client;
Server server;

if (args.Length == 0)
{
    PrintFormatDetails();
    return;
}

switch (args[0])
{
    case "-cs":
        client = new Client();
        client.SendFiles(args);

        break;

    case "-cr":
        client = new Client();
        client.RequestFiles(args);

        break;

    case "-s":
        server = new Server();
        server.Run(args);

        break;

    default:
        PrintFormatDetails();

        break;
}

void PrintFormatDetails()
{
        Console.WriteLine("Format:");
        Console.WriteLine();
        Console.WriteLine("-cs (Client Send)    [Server IP] [Server
           Port] [File1] t/o (t for text file, o for other file)
           [File2] ...");
        Console.WriteLine("-cr (Client Receive) [Server IP] [Server
           Port] [File1] t/o (t for text file, o for other file)
           [File2] ...");
```

```
        Console.WriteLine("-s (Server) [Port] loop (enter if the server should be run on the loopback address)");
        Console.WriteLine();
        Console.WriteLine("Any details not provided will be requested interactively, but the initial flag is mandatory.");
        Console.WriteLine();
        Console.WriteLine("Note that providing any details via CLI flags depends on the previous details also being enetered: you cannot pass files as an argument without entering the server IP and port!");
}
```

Listing 20: Program.cs

```
using System.Net.Sockets;
using System.Security.Cryptography;

class Client
{
    private static readonly IReadOnlyList<string> _textFileExtensions = new string[]
    {
        "123","1st","600","602","890","a","ab2","ab3","abc","abw","yml","ynab","yum","zabw","zeg","zig","zrtf","zsh","zw"
    }; // large list of plaintext file extensions - most have been removed for this writeup for clarity
    private static readonly IReadOnlyList<string> _otherFileExtensions = new string[]
    {
        "exe","dll","so","bin","msi","app","dmg","zip","rar","7z","gz","tar","tgz","bz2","xz","iso","jpg","jpeg","png","gif"
    }; // large list of other file extensions - most have been removed for this writeup for clarity

    public void SendFiles(string[] args)
    {
        TcpClient client = SetupClient(args);
        NetworkStream stream = client.GetStream();

        List<string> textFiles = new List<string>();
        List<string> otherFiles = new List<string>();

        if (args.Length > 3)
        {
            SortFiles(textFiles, otherFiles, args);
        }
        else
        {
            Console.WriteLine("Enter file paths to send (blank line to finish)");
            GetFileInput(textFiles, otherFiles);
```

```
31          }
32
33          using (client)
34          using (stream)
35          {
36              Aes aes = SetupAes.Client(stream);
37
38              FileSender sender = new FileSender(stream, aes);
39
40              List<Tuple<byte[], string>> filesToSend = new
                    List<Tuple<byte[], string>>();
41
42              foreach (string path in textFiles)
43              {
44                  string fileText = "";
45
46                  try
47                  {
48                      fileText = File.ReadAllText(path);
49                  }
50                  catch
51                  {
52                      Console.WriteLine(path + " is invalid, so not
                            sent!");
53                      continue;
54                  }
55
56                  filesToSend.Add(new Tuple<byte[], string>
57                          (HuffmanEncoder.WriteCompressedFile(fileText),
                                path + ".huff"));
58              }
59
60              foreach (string path in otherFiles)
61              {
62                  byte[] fileBytes;
63
64                  try
65                  {
66                      fileBytes = File.ReadAllBytes(path);
67                  }
68                  catch
69                  {
70                      Console.WriteLine(path + " is invalid, so not
                            sent!");
71                      continue;
72                  }
73
74                  filesToSend.Add(new Tuple<byte[], string>(fileBytes,
                        path));
75              }
76
```

```
77              sender.SendInt(filesToSend.Count); // no. of files to
                    expect
78
79              foreach (Tuple<byte[], string> file in filesToSend) //
                    sent after so non-existent files don't contribute to
                    count
80              {
81                  Console.WriteLine("Sending " + file.Item2);
82                  sender.SendFile(file.Item1, file.Item2);
83              }
84
85              sender.Finish();
86          }
87      }
88
89      public void RequestFiles(string[] args)
90      {
91          TcpClient client = SetupClient(args);
92          NetworkStream stream = client.GetStream();
93
94          using (client)
95          using (stream)
96          {
97              Aes aes = SetupAes.Client(stream);
98              FileSender sender = new FileSender(stream, aes);
99
100             sender.SendInt(0); // indicates desire to receive files
101
102             List<string> textFiles = new List<string>();
103             List<string> otherFiles = new List<string>();
104
105             if (args.Length > 3)
106             {
107                 SortFiles(textFiles, otherFiles, args);
108             }
109             else
110             {
111                 Console.WriteLine("Enter file paths to receive (blank
                        line to finish)");
112                 GetFileInput(textFiles, otherFiles);
113             }
114
115             string paths = "";
116
117             foreach (string path in textFiles) paths += path + ".huff"
                    + ",";
118             foreach (string path in otherFiles) paths += path + ",";
119
120             paths = paths.Remove(paths.Length - 1);
121
122             sender.SendData(paths);
```

```
123
124             sender.Finish();
125
126             FileReceiver receiver = new FileReceiver(stream, aes);
127
128             for (int i = 0; i < textFiles.Count + otherFiles.Count;
                    i++)
129             {
130                 receiver.GetFile("ClientReceived", true);
131             }
132         }
133     }
134
135     private TcpClient SetupClient(string[] args)
136     {
137         string serverIp = "";
138         int serverPort;
139
140         if (args.Length > 1)
141         {
142             serverIp = args[1];
143         }
144         if (args.Length <= 1 || !ValidateIP(serverIp)) // write this
                please
145         {
146             do
147             {
148                 Console.Write("Enter the server's IP address: ");
149                 serverIp = Console.ReadLine() ?? "127.0.0.1";
150             } while (!ValidateIP(serverIp));
151         }
152
153         if (!(args.Length > 2 && int.TryParse(args[2], out
                serverPort)))
154         {
155             Console.Write("Enter the server's port: ");
156
157             while (!int.TryParse(Console.ReadLine(), out serverPort))
158             {
159                 Console.WriteLine("Invalid port! It must be an
                        integer! Try again.");
160             }
161         }
162
163         try
164         {
165             return new TcpClient(serverIp, serverPort);
166         }
167         catch
168         {
169             Console.WriteLine("Error! Server not reached!");
```

```
170             return SetupClient(new string[]{}); // forces restart in
                    interactive mode
171         }
172     }
173 
174     private bool ValidateIP(string ipAddress)
175     {
176         if (!ipAddress.Contains('.')) return false;
177 
178         string[] splitIp = ipAddress.Split('.');
179 
180         if (splitIp.Length != 4) return false;
181 
182         foreach (string str in splitIp)
183         {
184             int ipPart;
185 
186             if (!int.TryParse(str, out ipPart)) return false;
187 
188             if (ipPart > 255 || ipPart < 0) return false;
189         }
190 
191         return true;
192     }
193 
194     private void GetFileInput(List<string> textFiles, List<string>
        otherFiles) // interactive
195     {
196         while (true)
197         {
198             string input = Console.ReadLine() ?? "";
199 
200             if (input == "") break;
201 
202             if (IsTextFile(input, true)) textFiles.Add(input);
203             else otherFiles.Add(input);
204 
205         }
206     }
207 
208     private void SortFiles(List<string> textFiles, List<string>
        otherFiles, string[] args) // non-interactive
209     {
210         for (int i = 3; i < args.Length; i++)
211         {
212             string fileName = args[i];
213             string flag = (i + 1 < args.Length) ? args[i+1].ToLower()
                    : "";
214 
215             if (flag == "t")
216             {
```

```
                textFiles.Add(fileName);
                i++;
                continue;
            }
            if (flag == "o")
            {
                otherFiles.Add(fileName);
                i++;
                continue;
            }

            if (IsTextFile(fileName, false)) textFiles.Add(fileName);
            else otherFiles.Add(fileName);
        }
    }

    private bool IsTextFile(string fileName, bool interactive)
    {
        string extension;

        if (!fileName.Contains('.')) extension = "";
        else extension = fileName.Split('.').Last();

        if (_textFileExtensions.Contains(extension)) return true;
        if (_otherFileExtensions.Contains(extension)) return false;

        if (!interactive) return false;

        Console.Write("Plaintext? (y/N) ");
        if ((Console.ReadLine() ?? "").ToLower() == "y") return true;

        return false;
    }
}
```

Listing 21: Client.cs

```
using System.Net;
using System.Net.Sockets;
using System.Security.Cryptography;

class Server
{
    public void Run(string[] args)
    {
        int port;

        if (args.Length < 2 || !int.TryParse(args[1], out port))
        {
            Console.WriteLine("Provide an integer port as the second
                argument!");
```

```
14              return;
15          }
16
17          IPAddress ip;
18          if (args.Length > 2 && args[2] == "loop") ip =
                IPAddress.Loopback;
19          else ip = IPAddress.Any;
20
21          TcpListener listener = new TcpListener(ip, port);
22          listener.Start();
23
24          Console.WriteLine("IP: " + ip + " Port: " + port);
25
26          Aes aes;
27
28          while (true)
29          {
30              using (NetworkStream stream =
                    listener.AcceptTcpClient().GetStream())
31              {
32                  aes = SetupAes.Server(stream);
33
34                  FileReceiver receiver = new FileReceiver(stream, aes);
35
36                  int fileNum = receiver.GetInt();
37
38                  for (int i = 0; i < fileNum; i++) // client sending
                        files
39                  {
40                      receiver.GetFile("ServerReceived");
41                  }
42
43                  if (fileNum == 0) // client retrieving files
44                  {
45                      string paths = receiver.GetString();
46
47                      if (paths == "")
48                      {
49                          Console.WriteLine("Corrupted transmission!");
50                          continue;
51                      }
52
53                      FileSender sender = new FileSender(stream, aes);
54
55                      foreach (string path in paths.Trim().Split(','))
56                      {
57                          try
58                          {
59                              byte[] fileBytes =
                                    File.ReadAllBytes("ServerReceived" +
                                    path);
```

```
60
61                              sender.SendFile(fileBytes, path);
62                          }
63                          catch
64                          {
65                              sender.SendFile(
66                                  System.Text.Encoding.ASCII.GetBytes
67                                  ("File not found!"), path);
68
69                              Console.WriteLine(path + " not found!");
70                          }
71                      }
72
73                      sender.Finish();
74                  }
75              }
76          }
77      }
78  }
```

Listing 22: Server.cs

3.2 Huffman Compression

```
1   class ByteMaker
2   {
3       private int[] _bits;
4       private int _bitIndex;
5       private int _trailingZeros;
6
7       private List<byte> _bytes;
8
9       public ByteMaker(bool isCompressed)
10      {
11          _bits = [isCompressed ? 1 : 0, 0, 0, 0, 0, 0, 0, 0]; // length
                8
12          _bitIndex = 4; // first bit indicates compressed, next 3 bits
                reserved to give data about trailing zeros
13          _trailingZeros = 0;
14
15          _bytes = new List<byte>();
16      }
17
18      public void Add(int bit)
19      {
20          if (bit != 1 && bit != 0) return;
21
22          _bits[_bitIndex++] = bit;
23
24          if (_bitIndex == 8) updateByte();
```

```
25      }
26
27      public void Add(List<int> bits)
28      {
29          foreach (int bit in bits) Add(bit);
30      }
31
32      public void Add(char character)
33      {
34          byte characterByte = (byte)character;
35
36          for (int i = 7; i >= 0; i--)
37          {
38              int bit = (characterByte >> i) & 1; // right-shift, then
                    AND with 1 to get "last" bit
39
40              Add(bit);
41          }
42      }
43
44      public byte[] Export()
45      {
46          updateByte();
47
48          _bytes[0] |= (byte)(_trailingZeros << 4); // push to bits 1-4,
                OR since already 0
49
50          return _bytes.ToArray();
51      }
52
53      private void updateByte()
54      {
55          byte newByte = new byte();
56
57          for (int i = 0; i < 8; i++)
58          {
59              newByte |= (byte)(_bits[i] << (7-i)); // left-shift, then
                    OR with existing byte
60          }
61
62          _bytes.Add(newByte);
63
64          _bits = [0, 0, 0, 0, 0, 0, 0, 0]; // length 8
65          _trailingZeros = 8 - _bitIndex;
66          _bitIndex = 0;
67      }
68  }
```

Listing 23: Huffman/ByteMaker.cs

```
1   class ByteReader
```

```
{
    private List<byte> _bytes;
    private int _bitIndex;
    private int _trailingNum;

    public ByteReader(byte[] bytes, out bool compressed)
    {
        _bytes = bytes.ToList();
        _bitIndex = 0;

        int firstBit = Read();

        if (firstBit == 0) compressed = false;
        else compressed = true;

        int a = Read();
        int b = Read();
        int c = Read();

        _trailingNum = a*4 + b*2 + c;
    }

    public int Read()
    {
        if (_bytes.Count() == 0) return -1;

        if (_bytes.Count() == 1 && _bitIndex >= 8 - _trailingNum)
            return -1; // don't read padding zeros

        int bit = (_bytes[0] >> (7 - _bitIndex++)) & 1;

        Update();

        return bit;
    }

    public char? ReadChar()
    {
        byte nextByte = 0;

        for (int i = 0; i < 8; i++)
        {
            int bit = Read();
            if (bit == -1) return null;

            if (bit == 1) nextByte |= (byte)(1 << (7-i)); // shift 1
                to its position and OR with exisiting 00...0 byte
        }

        return (char)nextByte;
    }
```

```
51
52      private void Update()
53      {
54          if (_bitIndex != 8) return;
55
56          _bytes.RemoveAt(0);
57          _bitIndex = 0;
58      }
59  }
```

Listing 24: Huffman/ByteReader.cs

```
1   static class HuffmanEncoder
2   {
3       public static byte[] WriteCompressedFile(string text)
4       {
5           HuffmanNode tree = GenerateTree(text);
6
7           // add encoding of tree
8           ByteMaker byteMaker = new ByteMaker(true);
9
10          Dictionary<char, List<int>> CharacterEncodings = new
                Dictionary<char, List<int>>();
11
12          SearchTree(tree, byteMaker, CharacterEncodings, new
                List<int>());
13          byteMaker.Add(0);
14
15          // add encoding of text
16          foreach (char c in text) byteMaker.Add(CharacterEncodings[c]);
17
18          byte[] bytes = byteMaker.Export();
19
20          if (bytes.Length > text.Length)
21          {
22              ByteMaker textByteMaker = new ByteMaker(false);
23
24              foreach(char c in text) textByteMaker.Add(c);
25
26              bytes = textByteMaker.Export();
27          }
28
29          return bytes;
30      }
31
32      private static HuffmanNode GenerateTree(string text)
33      {
34          // link characters to their count
35          Dictionary<char, int> CharacterCounts = new Dictionary<char,
                int>();
36
```

```
37          foreach (char c in text)
38          {
39              if (CharacterCounts.ContainsKey(c)) CharacterCounts[c]++;
40              else CharacterCounts[c] = 1;
41          }
42
43          // place characters in priorityqueue (based on freq) to
                generate the tree
44          PriorityQueue<HuffmanNode, int> NodeQueue = new
                PriorityQueue<HuffmanNode, int>(); // "base" trees of size 1
45
46          foreach (KeyValuePair<char, int> kvp in CharacterCounts)
47          {
48              NodeQueue.Enqueue(new HuffmanNode(kvp.Key, kvp.Value),
                    kvp.Value);
49          }
50
51          HuffmanNode tree = MergeNodes(NodeQueue); // tree = top node
                of tree
52
53          return tree;
54      }
55
56      // recursively merge nodes to make a tree
57      private static HuffmanNode MergeNodes(PriorityQueue<HuffmanNode,
            int> trees)
58      {
59          if (trees.Count == 1) return trees.Dequeue();
60
61          HuffmanNode tree1 = trees.Dequeue();
62          HuffmanNode tree2 = trees.Dequeue();
63
64          int newFreq = tree1.GetFrequency() + tree2.GetFrequency();
65
66          HuffmanNode newTree = new HuffmanNode(tree1, tree2, newFreq);
67
68          trees.Enqueue(newTree, newFreq);
69
70          return MergeNodes(trees);
71      }
72
73      // fill byteMaker with encoded tree, and CharacterEncodings with
            bits encoding characters, uses post order DFS
74      private static void SearchTree(HuffmanNode tree, ByteMaker
            byteMaker, Dictionary<char, List<int>> CharacterEncodings,
            List<int> current)
75      {
76          if (tree == null) return;
77
78          SearchTree(tree.GetLeft(), byteMaker, CharacterEncodings,
                current.Append(0).ToList());
```

```
79          SearchTree(tree.GetRight(), byteMaker,  CharacterEncodings,
                current.Append(1).ToList());
80
81          if (tree.HasCharacter()) // is leaf
82          {
83              byteMaker.Add(1);
84              byteMaker.Add(tree.GetCharacter());
85
86              CharacterEncodings[tree.GetCharacter()] = current;
87          }
88          else
89          {
90              byteMaker.Add(0);
91          }
92      }
93  }
```

Listing 25: Huffman/HuffmanEncoder.cs

```
1   static class HuffmanDecoder
2   {
3       public static void WriteRawFile(byte[] bytes, string path)
4       {
5           bool fileCompressed;
6
7           ByteReader byteReader = new ByteReader(bytes, out
                fileCompressed);
8           string rawFile = "";
9
10          if (!fileCompressed)
11          {
12              while (true)
13              {
14                  char? nextChar = byteReader.ReadChar();
15
16                  if (nextChar == null) break;
17
18                  rawFile += nextChar;
19              }
20          }
21          else
22          {
23              Stack<HuffmanNode> NodeStack = new Stack<HuffmanNode>();
24
25              while (true)
26              {
27                  int nextBit = byteReader.Read();
28
29                  if (nextBit == 1)
30                  {
31                      char? nextChar = byteReader.ReadChar();
```

```
                    if (nextChar == null) break;

                    NodeStack.Push(new HuffmanNode((char)nextChar));
                }
                else
                {
                    if (NodeStack.Count() == 1) break;

                    HuffmanNode right = NodeStack.Pop();
                    HuffmanNode left = NodeStack.Pop();

                    NodeStack.Push(new HuffmanNode(left, right));
                }
            }

            HuffmanNode tree = NodeStack.Pop(); // there should be
                just one node left

            Dictionary<string, char> EncodingCharacters = new
                Dictionary<string, char>(); // maps encodings to
                character
            GenerateEncodings(tree, EncodingCharacters, "");

            string currentEncoding = ""; // string, not int array, as
                arrays/lists compared by instance, not contents

            while (true)
            {
                int nextBit = byteReader.Read();
                if (nextBit == -1) break;

                currentEncoding += nextBit.ToString();

                if (!EncodingCharacters.ContainsKey(currentEncoding))
                    continue;

                rawFile += EncodingCharacters[currentEncoding];

                currentEncoding = "";
            }
        }

        try
        {
            File.WriteAllText(path, rawFile);
        }
        catch
        {
            Console.WriteLine("Error: Could not write to " + path);
        }
    }
```

```
78
79      private static void GenerateEncodings(HuffmanNode tree,
            Dictionary<string, char> EncodingCharacters, string current)
80      {
81          if (tree == null) return;
82
83          GenerateEncodings(tree.GetLeft(), EncodingCharacters, current
                + "0");
84          GenerateEncodings(tree.GetRight(), EncodingCharacters, current
                + "1");
85
86          if (tree.HasCharacter()) EncodingCharacters[current] =
                tree.GetCharacter();
87      }
88  }
```

Listing 26: Huffman/HuffmanDecoder.cs

```
1   class HuffmanNode
2   {
3       private char _character;
4       private int _frequency;
5       private HuffmanNode? _left;
6       private HuffmanNode? _right;
7
8       private bool _hasCharacter; // "isLeaf"
9
10      public HuffmanNode(char character, int frequency = 1)
11      {
12          _character = character;
13          _frequency = frequency;
14
15          _hasCharacter = true;
16      }
17
18      public HuffmanNode(HuffmanNode left, HuffmanNode right, int
            frequency = 1)
19      {
20          _left = left;
21          _right = right;
22          _frequency = frequency;
23
24          _hasCharacter = false;
25      }
26
27      public int GetFrequency() => _frequency;
28
29      public HuffmanNode? GetLeft() => _left;
30      public HuffmanNode? GetRight() => _right;
31
32      public bool HasCharacter() => _hasCharacter;
```

```
33      public char GetCharacter() => _character;
34  }
35
36  // NOTE: tree is just a "top-node" - links to left/right
```

Listing 27: Huffman/HuffmanNode.cs

3.3 Transmission Procedure

```
1   using System.Numerics;
2   using System.Net.Sockets;
3
4   static class Transmission
5   {
6       public static byte[] Serialise(byte[] bytes)
7       {
8           byte[] bytesLengthBytes = BitConverter.GetBytes(bytes.Length);
                // length 4 always
9
10          byte[] data =
                bytesLengthBytes.ToList().Concat(bytes.ToList()).ToArray();
11
12          return data;
13      }
14
15      public static byte[] Serialise(BigInteger bigInt)
16      {
17          byte[] bigIntBytes = bigInt.ToByteArray(true, true); //
                isUnsigned, isBigEndian
18
19          return Serialise(bigIntBytes);
20      }
21
22      public static byte[] Serialise(string data) =>
            Serialise(System.Text.Encoding.ASCII.GetBytes(data));
23
24      public static BigInteger ReadBigInt(NetworkStream stream) => new
            BigInteger(ReadByteArray(stream), true, true);
25
26      public static byte[] ReadByteArray(Stream stream)
27      {
28          int bytesLength = BitConverter.ToInt32(ReadBytes(stream, 4));
29
30          return ReadBytes(stream, bytesLength);
31      }
32
33      public static byte[] ReadBytes(Stream stream, int length)
34      {
35          byte[] bytes = new byte[length];
36
```

```
        int read = 0;
        while (read < length) read += stream.Read(bytes, read, length
            - read);

        return bytes;
    }
}
```

Listing 28: Transmission/Transmission.cs

```
using System.Net.Sockets;
using System.Security.Cryptography;

abstract class Transferer
{
    protected CryptoStream _cryptoStream;

    public Transferer(NetworkStream stream, Aes aes, CryptoStreamMode
        mode)
    {
        aes.Padding = PaddingMode.Zeros;

        _cryptoStream = new CryptoStream(stream, mode ==
            CryptoStreamMode.Write ? aes.CreateEncryptor() :
            aes.CreateDecryptor(), mode, leaveOpen: true);
    }
}

class FileSender : Transferer
{
    public FileSender(NetworkStream stream, Aes aes) : base(stream,
        aes, CryptoStreamMode.Write) { }

    public void SendFile(byte[] fileBytes, string fileName)
    {
        _cryptoStream.Write(
                Transmission.Serialise(Hashing.Hash(fileBytes))
            );

        _cryptoStream.Write(Transmission.Serialise(fileName));

        SendData(fileBytes);
    }

    public void SendData(string data)
    {
        byte[] bytes = System.Text.Encoding.ASCII.GetBytes(data);

        _cryptoStream.Write(
                Transmission.Serialise(Hashing.Hash(bytes))
            );
```

```
38
39          SendData(bytes);
40      }
41
42      public void SendInt(int data) // different name since no hashing
43      {
44          byte[] bytes = BitConverter.GetBytes(data); // 4 bytes
45
46          SendData(bytes);
47      }
48
49      private void SendData(byte[] bytes)
50      {
51          _cryptoStream.Write(Transmission.Serialise(bytes));
52          _cryptoStream.Flush();
53      }
54
55      public void Finish() => _cryptoStream.FlushFinalBlock();
56  }
57
58  class FileReceiver : Transferer
59  {
60      public FileReceiver(NetworkStream stream, Aes aes) : base(stream,
            aes, CryptoStreamMode.Read) { }
61
62      public void GetFile(string location, bool decompress = false)
63      {
64          byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
                // 32 byte hash read
65          byte[] nameBytes = Transmission.ReadByteArray(_cryptoStream);
                // read file name
66          byte[] fileBytes = Transmission.ReadByteArray(_cryptoStream);
67
68          if (Hashing.VerifyHash(fileBytes, hashBytes))
69          {
70              string name =
                    System.Text.Encoding.ASCII.GetString(nameBytes);
71
72              string filePath = location + name;
73
74              Directory.CreateDirectory(Path.GetDirectoryName(filePath)
                    ?? "");
75
76              if (decompress && name.Split('.').LastOrDefault() ==
                    "huff")
77              {
78                  HuffmanDecoder.WriteRawFile(fileBytes,
                        filePath.Substring(0, filePath.Length - 5));
79              }
80              else File.WriteAllBytes(filePath, fileBytes);
81          }
```

```
82          else Console.WriteLine("Corrupted transmission!");
83      }
84
85      public string GetString()
86      {
87          byte[] hashBytes = Transmission.ReadByteArray(_cryptoStream);
88          byte[] stringBytes = Transmission.ReadByteArray(_cryptoStream);
89
90          if (Hashing.VerifyHash(stringBytes, hashBytes))
91          {
92              return System.Text.Encoding.ASCII.GetString(stringBytes);
93          }
94
95          return "";
96      }
97
98      public int GetInt()
99      {
100         byte[] intBytes = Transmission.ReadByteArray(_cryptoStream);
101
102         return BitConverter.ToInt32(intBytes);
103     }
104 }
```

Listing 29: Transmission/Transferer.cs

```
1  using System.Security.Cryptography;
2  using System.Numerics;
3
4  static class RSA
5  {
6      public static Tuple<BigInteger, BigInteger, BigInteger>
           GenerateKeys()
7      {
8          BigInteger e = 65537;
9
10         BigInteger p = RSA.GenerateRSAPrime(e);
11         BigInteger q = RSA.GenerateRSAPrime(e);
12
13         BigInteger n = p*q;
14         BigInteger phin = (p-1)*(q-1);
15
16         BigInteger d = RSA.GetModMultInverse(e, phin);
17
18         return new Tuple<BigInteger, BigInteger, BigInteger>(n,e,d);
19     }
20
21     private static BigInteger GenerateRSAPrime(BigInteger e) // e must
           be prime in this implementation!
22     {
23         while (true)
```

```
24          {
25              BigInteger bigInt;
26
27              do bigInt = GenerateOdd512BitInt();
28              while (bigInt % e == 1); // ensures e will be relatively
                    prime with phi(n), as e used is prime
29
30              if (MillerRabin(bigInt, 40)) return bigInt;
31          }
32      }
33
34      private static BigInteger GetModMultInverse(BigInteger a,
            BigInteger m) // uses the extended euclidian algorithm to find
            x s.t. ax = 1 (mod m)
35      {
36          BigInteger r_0 = a;
37          BigInteger s_0 = 1;
38          BigInteger t_0 = 0;
39
40          BigInteger r_1 = m;
41          BigInteger s_1 = 0;
42          BigInteger t_1 = 1;
43
44          BigInteger r_2, s_2, t_2;
45
46          do
47          {
48              BigInteger q = r_0 / r_1;
49
50              r_2 = r_0 - q * r_1;
51              s_2 = s_0 - q * s_1;
52              t_2 = t_0 - q * t_1;
53
54              (r_0, r_1) = (r_1, r_2);
55              (s_0, s_1) = (s_1, s_2);
56              (t_0, t_1) = (t_1, t_2);
57
58          } while (r_2 != 0);
59
60          BigInteger d = s_0 % m;
61          if (d < 0) d += m;
62
63          return d;
64      }
65
66      private static BigInteger GenerateOdd512BitInt()
67      {
68          byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
69          RandomNumberGenerator.Fill(randomInt);
70
71          randomInt[0] |= (byte)(128); // sets MSB to 1, so num is 512
```

```
            bits

        randomInt[63] |= (byte)(1); // sets LSB to 1 - odd

        return new BigInteger(randomInt, true, true); // isUnsigned,
            isBigEndian
    }

    private static BigInteger GenerateInRange(BigInteger lower,
        BigInteger upper)
    {
        BigInteger num = -1;

        while (num < lower || num > upper)
        {
            byte[] randomInt = new byte[64]; // 512 bits = 64 bytes
            RandomNumberGenerator.Fill(randomInt);

            num = new BigInteger(randomInt, true, true);
        }

        return num;
    }

    private static bool MillerRabin(BigInteger n, int rounds)
    {
        int s = 1;
        while ((n-1) % (BigInteger)Math.Pow(2, s+1) == 0) s++;

        BigInteger d = (n-1)/(BigInteger)(Math.Pow(2,s));

        for (int i = 0; i < rounds; i++)
        {
            BigInteger a = GenerateInRange(2, n-3);

            BigInteger x = BigInteger.ModPow(a, d, n);

            for (int j = 0; j < s; j++)
            {
                BigInteger y = BigInteger.ModPow(x, 2, n);

                if (y == 1 && x != 1 && x != n-1) return false;

                x = y;
            }

            if (x != 1) return false;
        }

        return true;
    }
```

```
}
```

Listing 30: Transmission/RSA.cs

```
using System.Net.Sockets;
using System.Numerics;
using System.Security.Cryptography;

static class SetupAes
{
    public static Aes Client(NetworkStream stream)
    {
        BigInteger n = Transmission.ReadBigInt(stream);
        BigInteger e = Transmission.ReadBigInt(stream);

        Aes aes = Aes.Create(); // creates key and IV

        BigInteger plaintext = new BigInteger(aes.Key, true, true);

        BigInteger ciphertext = BigInteger.ModPow(plaintext, e, n);

        stream.Write(Transmission.Serialise(ciphertext)); // sends RSA
            encrypted AES key

        aes.IV = Transmission.ReadByteArray(stream);

        return aes;
    }

    public static Aes Server(NetworkStream stream)
    {
        Tuple<BigInteger, BigInteger, BigInteger> keys =
            RSA.GenerateKeys(); // n, e, d

        stream.Write(Transmission.Serialise(keys.Item1)); // send n
        stream.Write(Transmission.Serialise(keys.Item2)); // send e

        BigInteger ciphertext = Transmission.ReadBigInt(stream); //
            recieve RSA encrypted AES key

        BigInteger plaintext = BigInteger.ModPow(ciphertext,
            keys.Item3, keys.Item1); // M = C^d (mod n)

        byte[] aesKey = plaintext.ToByteArray(true, true); //
            isUnsigned, isBigEndian

        Aes aes = Aes.Create();
        aes.Key = aesKey;

        aes.GenerateIV();
        stream.Write(Transmission.Serialise(aes.IV)); // send IV
```

```
43
44          return aes;
45      }
46  }
```

Listing 31: Transmission/SetupAes.cs

```
1   using System.Security.Cryptography;
2
3   static class Hashing
4   {
5       public static byte[] Hash(byte[] messageBytes)
6       {
7           byte[] hashBytes;
8
9           using (SHA256 HashMaker = SHA256.Create())
10          {
11              hashBytes = HashMaker.ComputeHash(messageBytes);
12          }
13
14          return hashBytes;
15      }
16
17      public static bool VerifyHash(byte[] messageBytes, byte[] hash)
18      {
19          byte[] originalHashed = Hash(messageBytes);
20
21          if (originalHashed.Length != hash.Length) return false;
22
23          for (int i = 0; i < hash.Length; i++)
24          {
25              if (originalHashed[i] != hash[i]) return false;
26          }
27
28          return true;
29      }
30  }
```

Listing 32: Transmission/Hashing.cs

References

[1] Billy B. Brumley et al. "Practical realisation and elimination of an ECC-Related software bug attack". In: *Proceedings of the 12th Conference on Topics in Cryptology*. CT-RSA'12. San Francisco, CA: Springer-Verlag, 2012, pp. 171–186. ISBN: 9783642279539. DOI: 10.1007/978-3-642-27954-6_11. URL: https://doi.org/10.1007/978-3-642-27954-6_11.

[2] kartik. *RSA Algorithm in Cryptography*. 23rd July 2025. URL: https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/ (visited on 26/10/2025).

[3] Andrew D. Ker. *Computer Security Lecture Notes*. https://www.cs.ox.ac.uk/andrew.ker/docs/computersecurity-lecture-notes-mt2014.pdf. 2014.

[4] Professor Vijay Raghunathan. *ECE264: Huffman Coding*. 2017. URL: https://engineering.purdue.edu/ece264/17au/hw/HW13?alt=huffman (visited on 10/11/2025).

[5] Wikipedia contributors. *Miller–Rabin primality test — Wikipedia, The Free Encyclopedia*. [Online; accessed 16-February-2026]. 2025. URL: https://en.wikipedia.org/w/index.php?title=Miller%E2%80%93Rabin_primality_test&oldid=1320502394.

[6] Wikipedia contributors. *Extended Euclidean algorithm — Wikipedia, The Free Encyclopedia*. [Online; accessed 16-February-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Extended_Euclidean_algorithm&oldid=1335653676.